



Specifica Tecnica

Gruppo Coderius

`coderius01@gmail.com`

Versione 0.9.0

Tabella di versionamento

Versione	Data	Autore	Verificatore	Descrizione
0.9.0	2026/09/01	Ines Iadadi		Modifica alla struttura del documento e aggiornamento della sezione 2
0.8.0	2026/08/28	Alberto Canavese		Stesura della sezione 4 - Design pattern
0.7.0	2026/08/22	Ines Iadadi		Stesura della sezione Backend _G 3.6
0.6.0	2026/08/13	Filippo Zonta Rocha		Stesura della sezione 3.5
0.5.0	2026/08/10	Edis Hodja		Stesura della sezione 3.4
0.4.0	2026/07/29	Leonardo Lorenzin	Edis Hodja	Stesura iniziale della sezione 3
0.3.0	2026/07/24	Giovanni Bronte	Leonardo Lorenzin	Stesura della sezione 2
0.2.0	2026/07/24	Alberto Canavese	Leonardo Lorenzin	Stesura della sezione 1
0.1.0	2026/07/21	Giovanni Bronte	Leonardo Lorenzin	Prima stesura del documento

Indice

1. Introduzione	1
1.1. Scopo del documento	1
1.2. Scopo del prodotto	1
1.3. Glossario	3
1.4. Riferimenti	3
1.4.1. Riferimenti normativi	3
1.4.2. Riferimenti informativi	3
1.4.3. Documentazione delle tecnologie:	3
2. Tecnologie	4
2.1. Linguaggi di programmazione	4
2.2. Framework	4
2.3. Librerie	4
2.4. Strumenti di build e sviluppo	6
2.5. Persistenza dei dati	6
2.6. Deployment	7
2.7. Tecnologie per testing	8
3. Architettura del sistema	8
3.1. Architettura generale	8
3.1.1. Architettura logica	8
3.1.2. Modalità di comunicazione	9
3.1.3. Architettura di deployment	9
3.2. Stile architetturale	10
3.3. Pattern architetturali adottati	10
3.3.1. Client-Server	10
3.3.2. Layered Architecture	11
3.3.3. REST stateless	11
3.3.4. Repository	11
3.3.5. Dependency Injection	11
3.4. Motivazioni della scelta architetturale	12
3.4.1. Architettura a livelli anziché esagonale	12
3.4.2. Ripartizione delle responsabilità secondo il possesso del dato	12
3.4.3. Assenza di una base di dati	13
3.4.4. Interfaccia REST stateless	13
3.4.5. Monolite containerizzato anziché microservizi	13
3.5. Limiti dell'architettura	13
3.6. Design pattern	14
3.6.1. Criteri di scelta	14
3.6.2. Pattern architetturali	15
3.6.3. Pattern di progettazione adottati	15
3.6.4. Observer	18

3.6.5. Tabella riassuntiva	20
4. Decomposizione del sistema	21
4.1. Diagramma dei package	21
4.2. Frontend _G	21
4.2.1. Organizzazione a livelli	21
4.2.2. Presentation Layer	21
4.2.3. Application Layer	22
4.2.4. State Management	23
4.2.5. Infrastructure Layer	24
4.2.6. Domain Layer	24
4.2.7. Flussi applicativi	25
4.3. Backend _G	26
4.3.1. Organizzazione a livelli	26
4.3.2. Presentation Layer	26
4.3.3. Application Layer	26
4.3.4. Domain Layer	27
4.3.5. Persistence Layer	27
4.3.6. Endpoint REST	27
4.3.7. Testing	28
5. Diagrammi delle classi	28
5.1. Criteri di rappresentazione	28
5.2. Dominio frontend _G	28
5.2.1. Device	29
5.2.2. Asset _G	29
5.2.3. DecisionTree	30
5.2.4. Node, QuestionNode e LeafNode	31
5.2.5. Session	32
5.3. Dominio backend _G	33
5.4. Corrispondenza fra i due domini	33
5.5. Vincoli e regole di integrità	34
6. Tracciamento dei requisiti _G	35
6.1. Requisiti _G funzionali obbligatori	35
6.2. Requisiti _G funzionali desiderabili	39
6.3. Requisiti _G funzionali opzionali	41
7. Requisiti _G di sistema	43
7.1. Requisiti _G hardware	43
7.2. Requisiti _G software	44
7.3. Requisiti _G di rete	44

Elenco delle Figure

Figura 1	Diagramma delle pagine	22
Figura 2	Diagramma di sequenza dell'avanzamento nell'albero decisionale	26

1. Introduzione

1.1. Scopo del documento

Il presente documento descrive l'architettura logica e infrastrutturale del sistema sviluppato dal gruppo Coderius per il capitolato C1 *Automated EN 18031 Compliance Verification*.

Nel dettaglio, gli obiettivi preposti per questo documento sono:

- **Motivare l'infrastruttura tecnologica:** argomentare le ragioni dietro la scelta degli strumenti di sviluppo, selezionati per rispondere appieno ai criteri di flessibilità, performance e scalabilità richiesti dal progetto.
- **Strutturare la logica applicativa e i design pattern:** mappare l'organizzazione interna dei moduli e i loro flussi di comunicazione, dimostrando come i pattern implementati favoriscano la creazione di codice disaccoppiato e orientato al testing.
- **Pianificare la strategia di deployment:** definire le modalità di distribuzione dell'applicativo e l'allocazione delle sue parti all'interno degli ambienti operativi.
- **Promuovere la manutenibilità a lungo termine:** fornire direttive chiare al team di sviluppo per rendere la base di codice facilmente interpretabile, supportando così agevolmente le future iterazioni, correzioni ed espansioni del sistema.

1.2. Scopo del prodotto

Il prodotto mira ad automatizzare e digitalizzare il processo di verifica della conformità dei dispositivi radio alla normativa di sicurezza EN 18031, come richiesto dal capitolato **Automated EN18031 Compliance Verification** proposto da Bluewind S.r.l. L'obiettivo è sostituire le attuali procedure manuali, spesso dispendiose in termini di tempo e soggette a errore umano, con una soluzione software moderna ed efficiente.

Le funzioni principali offerte dal prodotto sono le seguenti:

- **Definizione del dispositivo:** il sistema consente di descrivere il dispositivo da valutare e gli asset_G che lo compongono, creandoli tramite interfaccia web oppure importandoli da file nei formati JSON e CSV. Per ciascun asset_G il sistema deriva automaticamente i requisiti_G della norma applicabili sulla base della sua tipologia.
- **Esecuzione dei decision tree_G:** per ogni coppia asset_G-requisito il sistema esegue l'albero decisionale corrispondente, riconducendo l'interazione a una sequenza di domande a risposta binaria affiancata dalla rappresentazione grafica dell'albero, che evidenzia il nodo_G corrente e il percorso già intrapreso.
- **Gestione della sessione di valutazione:** lo stato della valutazione può essere salvato su file in formato JSON in qualsiasi momento e ripreso successivamente dal punto in cui era stato interrotto. È inoltre possibile riaprire un requisito già valutato insieme a quelli che da esso dipendono.

- **Consultazione e esportazione degli esiti:** gli esiti dei singoli requisiti_G sono aggregati a livello di asset_G e di dispositivo e restano risalibili fino alla sequenza di domande e risposte che li ha determinati. Il sistema produce inoltre un report di conformità finale in formato PDF, contenente per ciascuna coppia asset_G-requisito l'esito e il percorso logico seguito.
- **Gestione del catalogo dei decision tree_G:** il sistema mette a disposizione un catalogo degli alberi decisionali disponibili, consultabili in forma grafica, ampliabile importando nuovi alberi in formato JSON o CSV ed esportabile nei medesimi formati.

Il prodotto è concepito come strumento interno alla proponente_G, opera interamente in locale e non prevede autenticazione né archiviazione centralizzata. Lo scambio dei dati fra installazioni viene affidato all'esportazione e all'importazione di file.

1.3. Glossario

Al fine di evitare ambiguità e incomprensioni legate alla terminologia tecnica presente in questo documento e nel resto della documentazione prodotta dal gruppo Coderius, è stato redatto un apposito glossario.

Ogni termine tecnico o di dominio che necessita di ulteriori chiarimenti è contrassegnato nel testo da una «G» a pedice (es. *termine_G*). Cliccando sull'indicatore, il lettore verrà reindirizzato direttamente alla definizione corrispondente all'interno del documento

Glossario.

1.4. Riferimenti

1.4.1. Riferimenti normativi

- [Capitolato C1 - Automated EN18031_G Compliance Verification, Bluewind S.r.l.](#)
- [Norme di Progetto - Coderius Group](#)
- [Slide IS A.A. 2025/2026 - Regolamento del progetto didattico](#)
- Norma EN 18031 — *Common security requirements for radio equipment.*

1.4.2. Riferimenti informativi

- [Slide IS A.A. 2023/2024 - Diagrammi delle Classi](#)
- [Slide IS A.A. 2022/2023 - Pattern Architetturali](#)
- [Slide IS A.A. 2022/2023 - Design Pattern Creazionali](#)
- [Slide IS A.A. 2022/2023 - Design Pattern Strutturali](#)
- [Slide IS A.A. 2025/2026 - Progettazione Software](#)

1.4.3. Documentazione delle tecnologie:

- [Python 3 Official Documentation](#)
- [TypeScript Official Documentation](#)
- [React Official Documentation](#)
- [Vite Getting Started Guide](#)
- [Flask Official Documentation](#)
- [Docker Documentation](#)
- [Vitest Documentation](#)
- [React Testing Library Documentation](#)

2. Tecnologie

Nelle seguente sezione vengono descritte le tecnologie usate per lo sviluppo del capitolato **Automated EN18031_G Compliance Verification**.

2.1. Linguaggi di programmazione

Tecnologia	Versione	Descrizione
Python	3.12	Linguaggio ad alto livello, interpretato e multi-paradigma, impiegato per la realizzazione del backend _G . La scelta risponde a un vincolo del capitolato, essendo il linguaggio prediletto dalla proponente _G , e trova conferma nella ricchezza della libreria standard e nella maturità dell'ecosistema di test.
TypeScript	6.0	Metalinguaggio open source sviluppato da Microsoft che si basa su JavaScript, aggiungendovi la tipizzazione statica. Nel nostro progetto è utilizzato per lo sviluppo del frontend _G assieme alla libreria React: l'introduzione dei tipi garantisce maggiore robustezza, facilita la manutenibilità del codice nel tempo e permette di intercettare numerosi errori già in fase di compilazione.

2.2. Framework

Tecnologia	Versione	Descrizione
Flask	3.1.3	Micro-framework Web scritto in Python, è progettato per essere rapido e semplice, offrendo al contempo la possibilità di essere utilizzato per applicazioni complesse. È inoltre molto semplice da imparare, flessibile e ha buone performance.
Flask-CORS	6.0	Estensione di Flask che gestisce le intestazioni <i>Cross-Origin Resource Sharing</i> . È necessaria in ambiente di sviluppo, dove client e server sono serviti da due origini distinte.
React	19.2	Libreria per la costruzione di interfacce utente a componenti, impiegata per l'intero frontend _G . Il modello dichiarativo e la composizione per componenti permettono di mantenere la logica di presentazione separata da quella applicativa, delegata a hook dedicati.
React Router	7.18	Libreria di instradamento lato client per React. Associa a ciascuna vista un percorso proprio e consente di applicare guardie di accesso alle pagine che richiedono precondizioni, come l'esistenza di una sessione di valutazione attiva.

2.3. Librerie

Tecnologia	Versione	Descrizione
------------	----------	-------------

React	19.2.7	React è una libreria completa, molto flessibile e permette la creazione e la gestione dell'interfaccia utente. Nel nostro progetto viene utilizzato per le varie librerie che permettono di gestire l'interfaccia utente al meglio.
React Flow	12.11	Libreria per la rappresentazione interattiva di grafi e diagrammi a nodi in applicazioni React, distribuita nel pacchetto <code>@xyflow/react</code> . Nel prodotto è impiegata per la visualizzazione del decision tree _G , con i nodi resi come componenti React personalizzati e supporto nativo a spostamento e ingrandimento della vista. La libreria non determina la disposizione dei nodi, che resta affidata alla regola pura <code>treeLayout</code> del Domain Layer: riceve le posizioni già calcolate e si occupa della sola resa grafica. Poiché la rappresentazione è di sola consultazione, spostamento e connessione dei nodi sono disabilitati.
React PDF	4.9	Libreria per la generazione di documenti PDF a partire da componenti React, distribuita nel pacchetto <code>@react-pdf/renderer</code> . Nel prodotto è impiegata per la produzione del report di conformità finale, la cui struttura è dichiarata come componente <code>ReportDocument</code> e resa in un documento scaricabile dall'utente. La generazione avviene interamente nel client, coerentemente con il criterio di possesso del dato illustrato in Sezione 3.4: il report si fonda esclusivamente su dispositivo e sessione, che risiedono nel client.
Zustand	5.0	Libreria minimale per la gestione dello stato globale in applicazioni React. Gli store si definiscono come funzioni hook, senza richiedere componenti <i>Provider</i> né il <i>boilerplate</i> di azioni e <i>reducer</i> tipico di altre soluzioni. Nel prodotto realizza i tre store separati per area funzionale (<code>DeviceStore</code> , <code>SessionStore</code> , <code>TreeStore</code>).
Zod	4.4	Libreria per la definizione e validazione di schemi dati, con inferenza automatica dei tipi TypeScript a partire dallo schema stesso. Nel nostro progetto viene utilizzata nel Domain Layer del frontend _G per validare la struttura dei dati in ingresso (dispositivi, asset _G) prima che vengano elaborati dal resto dell'applicazione.
react-hot-toast	2.6	Libreria leggera per la gestione di notifiche toast in applicazioni React, con supporto nativo a coda, timer di auto-dismiss e stacking dei messaggi. Nel nostro progetto viene utilizzata nell'Infrastructure Layer del frontend _G come

		implementazione concreta dietro l'interfaccia <code>NotificationService</code> .
TanStack Query	5.101	Libreria per la gestione dello stato di origine remota in applicazioni React, con supporto a memorizzazione dei risultati, deduplicazione delle richieste in volo e ritentativi. Nel prodotto è impiegata in modo imperativo tramite <code>queryClient</code>, senza montare alcun <i>provider</i>, per evitare che lo stesso <code>decision tree_G</code> venga richiesto più volte nel corso di una valutazione.

2.4. Strumenti di build e sviluppo

Tecnologia	Versione	Descrizione
Vite	8.2	Vite è diventato lo standard de facto dell'ecosistema frontend _G . È il motore ufficiale dietro i framework moderni come Vue, e i template ufficiali di React. È inoltre molto più veloce a creare il server rispetto ad altri strumenti simili ed è anche molto facile da implementare grazie alla sua configurazione minima richiesta.
Node.js	22	Ambiente di esecuzione JavaScript, impiegato per l'esecuzione degli strumenti di sviluppo e per la fase di build dell'immagine Docker del frontend _G .
ESLint	10.8	Analizzatore statico per JavaScript e TypeScript, configurato con <code>typescript-eslint</code> per l'applicazione delle regole di codifica stabilite dalle Norme di Progetto. È eseguito automaticamente nella pipeline di integrazione continua.
Ruff	/	Analizzatore statico e formattatore per Python. Nel prodotto è configurato con una lunghezza massima di riga di 100 caratteri ed è eseguito in integrazione continua nelle modalità <code>ruff check</code> e <code>ruff format --check</code> .

2.5. Persistenza dei dati

Il sistema non impiega alcun sistema di gestione di basi di dati. La scelta è coerente con la natura del dominio applicativo: l'unica collezione di dati posseduta dal sistema è il catalogo dei `decision treeG`, un insieme ristretto di documenti strutturati, di sola lettura durante l'esecuzione di una valutazione e privo di relazioni da interrogare.

Tecnologia	Versione	Descrizione
JSON	RFC 8259	Formato di scambio dati impiegato in tre ruoli distinti: rappresentazione dei <code>decision tree_G</code> nella cartella <code>backend_G/data/decision_trees/</code> , corpo delle richieste e delle risposte dell'API REST, e formato di esportazione e importazione di dispositivi e sessioni di valutazione.

CSV	RFC 4180	Formato tabellare impiegato come formato alternativo di scambio per i dispositivi e per i decision tree _G , secondo quanto richiesto dai requisiti _G RF-Ob05, RF-Ob21, RF-Ob79 e RF-D23.
-----	----------	--

Il dispositivo in lavorazione e la sessione di valutazione non sono mai scritti su disco dal server: risiedono nello stato del client per l'intera durata del lavoro e vengono esportati su file su richiesta dell'utente. Le implicazioni architetturali di questa scelta sono discusse nella sezione Sezione 3.

2.6. Deployment

Tecnologia	Versione	Descrizione
Docker	3.1.3	Piattaforma open source che consente agli sviluppatori di creare, implementare, eseguire, aggiornare e gestire i container. I container sono componenti standardizzati ed eseguibili che combinano il codice sorgente dell'applicazione con le librerie e le dipendenze del sistema operativo necessarie per eseguire tale codice in qualsiasi ambiente.
Docker Compose	/	Strumento di orchestrazione dei container. Il file <code>docker-compose.yml</code> descrive i due servizi, la rete privata che li collega, l'esposizione delle porte verso l'ospite (8080 per il frontend _G , 5000 per il backend _G) e il montaggio della cartella dei decision tree _G come volume.
Nginx	1.27	Server HTTP impiegato nell'immagine del frontend _G per servire i file statici prodotti dalla build e per inoltrare al backend _G le richieste dirette agli endpoint dell'API. La presenza del <i>reverse proxy</i> consente al client di rivolgersi a percorsi relativi, rendendo indirizzo e porta del backend _G un dettaglio della configurazione di deployment anziché del codice applicativo.

2.7. Tecnologie per testing

Tecnologia	Versione	Descrizione
Vitest	/	Vitest è un framework per il test di applicazioni TypeScript e JavaScript, progettato per essere veloce e leggero. Sfruttando la stessa configurazione di Vite, garantisce un ambiente di testing altamente performante e coerente con la build.
React Testing Library	/	Libreria per il testing di componenti React. Offre funzioni di utilità essenziali basate su react-dom e react-dom/test-utils, incoraggiando al contempo migliori pratiche di testing.
Pytest	/	Framework per il test di Python, che permette di scrivere test automatizzati in modo semplice e leggibile.

3. Architettura del sistema

3.1. Architettura generale

3.1.1. Architettura logica

Il sistema adotta il paradigma **Client-Server**, con una separazione netta fra la componente che gestisce l'interazione con l'utente e quella che ospita la logica applicativa e il possesso dei dati. Tenere distinte le due responsabilità consente di far evolvere ciascuna parte in modo autonomo e riduce le dipendenze reciproche fra le tecnologie impiegate.

Il prodotto si compone di due unità distribuibili:

- un **client** React, che costituisce l'intera interfaccia grafica: presenta le viste, raccoglie e valida gli input, conserva lo stato della valutazione in corso, esegue la navigazione lungo il decision tree_G e produce i file di esportazione;
- un **server** Flask scritto in Python 3.12, che espone un'API REST: valida i dati in ingresso, possiede il catalogo dei decision tree_G e ne governa il caricamento, l'importazione, l'esportazione e la derivazione dei requisiti_G applicabili a un tipo di asset_G.

La ripartizione delle responsabilità fra le due unità segue il criterio del possesso del dato, discusso nella Sezione 2.5. Il server possiede il catalogo dei decision tree_G, che mette a disposizione dell'utente; non possiede invece né il dispositivo né la sessione di valutazione, che l'utente porta al sistema e riporta via sotto forma di file. Ne consegue che la navigazione lungo l'albero avviene interamente nel client; rispondere a una domanda del decision tree_G non comporta alcuna richiesta al server, poiché l'albero è stato scaricato per intero all'ingresso nel requisito.

Al server ci si rivolge esclusivamente per ottenere un decision tree_G o l'elenco del catalogo, importare o esportare un decision tree_G, risolvere gli identificativi di dispositivo e asset_G e derivare i requisiti_G applicabili a un tipo di asset_G. Ogni altra operazione, come

l'avanzamento nell'albero, calcolo degli esiti aggregati, salvataggio e ripresa della sessione, esportazione del dispositivo, viene svolta nel client.

Internamente ciascuna unità è organizzata a livelli con dipendenza a senso unico, secondo quanto descritto in Sezione 4.

La collocazione del report nel client si fonda unicamente su dispositivo, sessione di valutazione e alberi dei requisiti_G esaminati, dati che risiedono nel client. Produrlo lato server avrebbe richiesto di trasmettere l'intera sessione a un componente che non la possiede, introducendo uno stato applicativo in contrasto con il carattere stateless dell'API. Non è stato pertanto introdotto alcun endpoint dedicato.

3.1.2. Modalità di comunicazione

Client e server dialogano esclusivamente attraverso richieste **HTTP** secondo lo stile architetturale **REST**, con i dati veicolati in formato JSON. Fa eccezione l'importazione di un decision tree_G, che trasmette il file come multipart/form-data, e la sua esportazione, che restituisce text/csv quando richiesto in tale formato.

Lo scambio è stateless in quanto il server non conserva alcuno stato di sessione fra una richiesta e la successiva, e ogni chiamata contiene tutte le informazioni necessarie alla propria elaborazione. Questa impostazione è dovuta al fatto che dispositivo e sessione risiedono nel client e di conseguenza non vi_G è alcuno stato conversazionale da mantenere lato server.

Il client non conosce l'indirizzo del server. FetchApiClient è costruito con un baseUrl vuoto e compone quindi percorsi relativi (/devices, /assets, /decision-trees), che raggiungono il backend_G grazie a un **reverse proxy**:

- in sviluppo è il server di Vite, la cui configurazione inoltra i percorsi dell'API a `http://localhost:5000`;
- in produzione è Nginx, che nell'immagine del frontend_G serve i file statici della build e inoltra i medesimi percorsi al servizio backend_G sulla rete privata dei container.

L'indirizzo e la porta del backend_G sono pertanto un dettaglio della configurazione di deployment e non compaiono nel codice applicativo. L'estensione Flask-CORS resta necessaria per gli scenari di esecuzione in cui client e server sono serviti da origini distinte.

Il client ignora completamente il modo in cui i dati sono archiviati sul server e il modo in cui la validità di un albero viene verificata: si attiene ai contratti stabiliti dagli endpoint, che costituiscono l'unico punto di contatto fra le due componenti e l'unica fonte di verità condivisa. Gli endpoint sono elencati in Sezione 4.3.6.

3.1.3. Architettura di deployment

Il sistema è distribuito come applicazione **monolitica containerizzata**: un solo servizio di backend_G e un solo servizio di frontend_G, orchestrati tramite Docker Compose e collegati da una rete privata.

I due servizi sono così configurati:

- **backend_G**: immagine derivata da `python:3.12-slim`, espone la porta 5000 e monta la cartella `backendG/data` come volume, cosicché i `decision treeG` importati dall'utente persistano fra i riavvii del container;
- **frontend_G**: immagine costruita in due fasi che espone la porta 80 del container sulla porta 8080 dell'ospite.

La scelta di un monolite in luogo di una scomposizione in microservizi è coerente con la natura del dominio applicativo: il sistema elabora un dispositivo alla volta all'interno di una singola sessione di valutazione, per un solo utente, senza richiedere né elaborazione concorrente di grandi volumi di richieste né scalabilità orizzontale indipendente delle parti. L'adozione di microservizi introdurrebbe una complessità infrastrutturale sproporzionata a fronte di un dominio ben delimitato.

La separazione fra client e server, unita all'organizzazione interna a livelli, è già sufficiente a garantire modularità e disaccoppiamento senza ricorrere a una scomposizione fisica. Un monolite containerizzato risulta inoltre più semplice da avviare e arrestare, richiede meno risorse computazionali e consente cicli di rilascio più rapidi: caratteristiche adatte sia alle dimensioni del gruppo, sia al carattere di strumento interno del prodotto.

3.2. Stile architetturale

Entrambe le unità adottano internamente un'**architettura a livelli** (*Layered Architecture*), con dipendenza a senso unico dai livelli superiori verso quelli inferiori. Il livello di dominio non dipende da alcun altro livello.

Si è deliberatamente evitata l'applicazione rigorosa di un'architettura esagonale. Il dominio applicativo è delimitato ad un solo meccanismo di persistenza su filesystem, una sola API REST esposta a un solo tipo di client. Introdurre porte e adattatori espliciti per ogni dipendenza tecnica aggiungerebbe indirezionalità senza un beneficio proporzionato in termini di sostituibilità. Il disaccoppiamento esplicito fra interfaccia e implementazione è adottato là dove esiste una ragione concreta per prevedere un'implementazione alternativa (l'accesso al catalogo dei `decision treeG`, la comunicazione HTTP, la presentazione delle notifiche) e omissio altrove.

Analogamente, lato client i service applicativi scrivono direttamente sugli store anziché passare da un'astrazione intermedia, mantenendo comunque il vincolo di dipendenza a senso unico.

3.3. Pattern architetturali adottati

3.3.1. Client-Server

Il pattern è realizzato dalla separazione fra il client React e il server Flask, eseguiti come container distinti e comunicanti tramite HTTP. È discusso in Sezione 3.1.1.

3.3.2. Layered Architecture

L'organizzazione interna di entrambe le unità in livelli con dipendenza a senso unico è il vincolo strutturale portante del prodotto. È ciò che permette di sostituire un dettaglio tecnico senza toccare la logica applicativa, ed è verificabile: i test del dominio non richiedono alcun contesto applicativo Flask né alcun componente React montato per essere eseguiti. La composizione dei livelli è descritta in Sezione 4.

3.3.3. REST stateless

Il server non mantiene stato di sessione fra le richieste. Ogni chiamata è autosufficiente, il che rende le rotte verificabili in isolamento e rimuove ogni vincolo di ordine fra le richieste.

3.3.4. Repository

Il pattern, appartenente al catalogo di Fowler, interpone fra Application Layer e sorgente dei dati un'interfaccia che espone le operazioni sulla collezione come se questa fosse disponibile in memoria.

Nel prodotto è applicato secondo un principio deliberatamente selettivo: un repository esiste solo per le entità che il backend_G possiede autonomamente, non per ogni entità di dominio. Il catalogo dei decision tree_G è l'unico dato che il sistema possiede e mette a disposizione dell'utente, e per esso esistono l'interfaccia `IDecisionTreeRepository` — che dichiara `get()`, `save()`, `delete()` e `list()` — e l'implementazione concreta `JsonDecisionTreeRepository`, che legge e scrive i file JSON in `backendG/data/decision_trees/`.

Device e Asset_G non dispongono di un repository: non sono collezioni possedute e servite per identificativo dal server, ma dati che l'utente porta al sistema e riporta via. Di conseguenza `POST /devices` è privo di effetti sul disco e il dispositivo vive per l'intera durata del lavoro nel solo `DeviceStore` del client.

Il beneficio è verificabile nel codice: i test di `DecisionTreeService` sostituiscono l'implementazione concreta con `FakeDecisionTreeRepository`, che realizza la medesima interfaccia mantenendo gli alberi in memoria, consentendo di collaudare la logica applicativa senza alcun accesso al disco.

3.3.5. Dependency Injection

Il principio prevede che un componente riceva i propri collaboratori dall'esterno anziché istanziarli, cosicché il grafo degli oggetti sia composto in un unico punto del sistema.

Lato **backend_G** il principio è applicato integralmente: `create_app()` è l'unico punto in cui compaiono i nomi delle classi concrete, e costruisce il repository, lo inietta nel `DecisionTreeService` e passa quest'ultimo alle factory dei blueprint. La direzione delle dipendenze è pertanto verificabile ispezionando un solo file.

Lato **frontend_G** l'applicazione è parziale, e costituisce il principale debito architetturale individuato:

- `DeviceService` istanzia direttamente `FetchApiClient` a livello di modulo;

- `DecisionTreeService` accetta un'implementazione di `ApiClientService` nel costruttore, ma ne dichiara una concreta come valore predefinito;
- `DecisionTreeService.exportTree()` invoca `fetch` direttamente, eludendo l'interfaccia `ApiClientService`.

Le prime due condizioni non compromettono la verificabilità ma introducono una dipendenza in fase di compilazione dalla classe concreta, in violazione del principio di inversione delle dipendenze. La terza è una violazione della regola secondo cui ogni comunicazione con il backend_G transita per `ApiClientService`.

3.4. Motivazioni della scelta architetturale

Le decisioni architetturali descritte nelle sezioni precedenti sono qui raccolte con la rispettiva motivazione, così da rendere esplicito il ragionamento che le ha prodotte e i vincoli entro cui sono state prese. Ciascuna è stata valutata rispetto alle alternative disponibili, non adottata per consuetudine.

3.4.1. Architettura a livelli anziché esagonale

L'architettura esagonale avrebbe imposto la definizione di porte e adattatori espliciti per ogni dipendenza tecnica. Il dominio applicativo è però delimitato: un solo meccanismo di persistenza, una sola API esposta a un solo tipo di client, nessuna previsione di sorgenti dati alternative. In tali condizioni l'introduzione sistematica di astrazioni aggiungerebbe indirezione senza variabilità da governare.

L'architettura a livelli garantisce gli stessi due benefici che qui interessano — separazione delle responsabilità e verificabilità del dominio in isolamento — a un costo strutturale inferiore. Il disaccoppiamento esplicito fra interfaccia e implementazione è stato adottato in modo **selettivo**, nei tre punti in cui esiste una ragione concreta per prevedere una realizzazione alternativa: l'accesso al catalogo dei decision tree_G, la comunicazione HTTP e la presentazione delle notifiche.

3.4.2. Ripartizione delle responsabilità secondo il possesso del dato

La ripartizione fra client e server non segue la separazione convenzionale fra presentazione e logica di business, ma il criterio del possesso del dato: il server ospita ciò che il sistema possiede e mette a disposizione dell'utente, il client ciò che l'utente porta al sistema e riporta via.

Il catalogo dei decision tree_G è dato del sistema; il dispositivo e la sessione di valutazione sono dell'utente, che li crea, li esporta su file e li reimporta. Ne discende che la navigazione lungo l'albero avviene interamente nel client e che il server non conserva né dispositivi né sessioni.

Il criterio produce due conseguenze verificabili: nessuna richiesta di rete è necessaria per rispondere a una domanda del decision tree_G, e l'utente conserva il pieno controllo dei propri dati, che non transitano né permangono su alcun archivio condiviso. Quest'ultima

proprietà è coerente con la natura di strumento interno del prodotto e con la decisione `VEG-7.1`.

3.4.3. Assenza di una base di dati

L'unica collezione posseduta dal sistema è il catalogo dei decision tree_G: un insieme ristretto di documenti strutturati, di sola lettura durante una valutazione e privo di relazioni da interrogare. Un sistema di gestione di basi di dati risolverebbe problemi che il prodotto non presenta — interrogazione relazionale, accesso concorrente in scrittura, integrità referenziale — introducendo un terzo servizio da avviare e configurare su ciascuna postazione.

La persistenza su file JSON mantiene il catalogo ispezionabile e modificabile con strumenti ordinari, proprietà utile in un contesto in cui gli alberi sono scambiati manualmente fra installazioni.

3.4.4. Interfaccia REST stateless

L'assenza di stato conversazionale lato server non è una scelta indipendente, ma la conseguenza diretta del criterio di possesso del dato: poiché dispositivo e sessione risiedono nel client, non esiste alcuno stato da mantenere fra una richiesta e la successiva. Ne deriva che ogni rotta è verificabile in isolamento e che non sussiste alcun vincolo di ordine fra le chiamate.

3.4.5. Monolite containerizzato anziché microservizi

Il sistema elabora un dispositivo alla volta, per un solo utente, senza elaborazione concorrente di grandi volumi né necessità di scalare le parti in modo indipendente. Una scomposizione in servizi introdurrebbe comunicazione fra servizi, individuazione dei servizi e coerenza dei dati distribuiti: complessità infrastrutturale priva di contropartita.

La separazione fra client e server, unita all'organizzazione interna a livelli, fornisce già la modularità necessaria senza richiedere una scomposizione fisica. La containerizzazione garantisce inoltre che l'ambiente di esecuzione sia il medesimo su ogni postazione, requisito rilevante per un prodotto avviato localmente da utenti che non sono sviluppatori.

3.5. Limiti dell'architettura

Le scelte descritte comportano limiti che si ritiene corretto dichiarare. Nessuno di essi compromette il soddisfacimento dei requisiti_G, ma ciascuno delimita il campo entro cui il prodotto può evolvere senza una revisione architetturale.

Il vincolo di dipendenza fra livelli è una disciplina, non una proprietà imposta. Né Flask né React impediscono a un modulo di un livello superiore di invocare direttamente un meccanismo tecnico: nulla vieta a una pagina di chiamare `fetch`. Il vincolo è presidiato dalle convenzioni di codifica e dalla revisione fra pari, non dagli strumenti. Che si tratti di un limite reale è dimostrato dal caso di `DecisionTreeService.exportTree()`, che invoca `fetch` eludendo l'interfaccia `ApiClientService`

La logica di valutazione risiede nel client. La navigazione dell'albero, il calcolo degli esiti aggregati e la gestione della sessione sono realizzati in TypeScript e non sono

raggiungibili dall'esterno. L'introduzione di un secondo tipo di client — un'applicazione mobile, un'interfaccia a riga di comando, un'integrazione con sistemi della proponente_G — richiederebbe di reimplementare tale logica oppure di spostarla lato server, con una revisione del criterio di possesso del dato.

Il modello di dominio è duplicato in due linguaggi. Le entità esistono in TypeScript e in Python senza generazione automatica dell'una dall'altra: la coerenza è presidiata unicamente dal contratto degli endpoint e dai test. Una divergenza fra le due rappresentazioni non è rilevata da alcuno strumento in fase di compilazione.

L'assenza di base di dati esclude l'interrogazione trasversale. Ogni operazione che percorra l'intero catalogo — come la derivazione dei requisiti_G applicabili a un tipo di asset_G — richiede la lettura di tutti i file. Il costo è trascurabile con alcune decine di documenti, ma l'introduzione di funzionalità di ricerca o di filtraggio sul catalogo imporrebbe di riconsiderare la decisione.

L'assenza di persistenza dei dati dell'utente sposta su di lui la responsabilità della conservazione. Un dispositivo o una sessione non esportati su file vanno perduti alla chiusura dell'applicazione. È una conseguenza accettata del criterio di possesso del dato e della decisione VE_G-7.1, non un difetto di realizzazione, ma va tenuta presente nella redazione del Manuale Utente.

Il monolite non consente scalabilità indipendente delle parti. La condizione è irrilevante nello scenario d'uso previsto, in cui un'installazione serve un solo utente per volta, ma lo diverrebbe qualora il prodotto fosse esposto come servizio condiviso.

3.6. Design pattern

3.6.1. Criteri di scelta

I pattern documentati in questa sezione non sono stati selezionati a priori da un catalogo, ma individuati a partire dai problemi concretamente emersi nella progettazione del prodotto. Ciascuno di essi risponde ad almeno uno dei seguenti obiettivi, coerenti con l'organizzazione a livelli descritta nelle sezioni precedenti:

- **Verificabilità:** consentire il collaudo della logica applicativa senza richiedere una rete attiva, un'interfaccia grafica montata o file reali sul disco.
- **Assenza di logica applicativa ai bordi:** mantenere pagine e rotte prive di decisioni, in coerenza con il vincolo di dipendenza a senso unico fra i livelli.
- **Estendibilità rispetto ai requisiti_G EN 18031:** permettere l'introduzione di nuovi requisiti_G e di nuovi formati di scambio senza riprogettare i moduli esistenti.
- **Unicità del punto di verità:** evitare la duplicazione di logiche di validazione o di trasformazione dei dati in punti diversi del sistema.

Si sottolinea che nessuna scelta è motivata dalla previsione di un'infrastruttura distribuita. L'applicazione opera interamente in locale e la condivisione dei decision tree_G modificati avviene all'esterno dell'applicativo, mediante esportazione e distribuzione manuale dei file.

Il perimetro di questa sezione è costituito dai pattern del catalogo di Gamma, Helm, Johnson e Vlissides che presentano attinenza con il problema affrontato, integrati dal solo Repository, che appartiene al catalogo architetturale di Martin Fowler ed è documentato in quanto criterio di accesso ai dati adottato dal backend_G. I pattern del catalogo privi di rapporto con il dominio applicativo non sono discussi. Ciò comporta che alcune tecniche di progettazione ampiamente impiegate nel prodotto non vi_G compaiano come pattern: è il caso dell'iniezione delle dipendenze, per cui ciascun componente riceve i propri collaboratori dal costruttore anziché istanziarli, e il grafo degli oggetti viene composto in un unico punto del sistema. Si tratta di un principio di progettazione e non di un pattern del catalogo, e come tale è descritto contestualmente al Factory Method, che ne costituisce il luogo di applicazione, anziché in una scheda propria.

Per ciascun pattern sono indicati il problema affrontato, la soluzione adottata e i moduli concretamente coinvolti. Tutti i pattern qui documentati sono realizzati nel codice del prodotto; la sezione si chiude con quelli che non trovano applicazione, corredati della motivazione tecnica che ne esclude l'adozione.

3.6.2. Pattern architetturali

I pattern architetturali che governano la struttura complessiva del sistema sono già stati motivati nelle sezioni precedenti e vengono qui richiamati per completezza:

- **Client-Server:** separazione tra il client React e il server Flask, entrambi eseguiti in locale come container distinti e coordinati da Docker Compose.
- **Layered Architecture:** organizzazione interna in livelli con dipendenza a senso unico, in sei livelli lato frontend_G e quattro lato backend_G.
- **REST stateless:** assenza di stato di sessione lato server; ogni richiesta contiene tutte le informazioni necessarie alla propria elaborazione.
- **Monolite containerizzato:** distribuzione come singolo backend_G e singolo frontend_G, in luogo di una scomposizione in servizi indipendenti.

3.6.3. Pattern di progettazione adottati

3.6.3.1. Adapter

- **Problema:** la logica applicativa incontra in due punti un meccanismo tecnico la cui interfaccia è espressa nei termini della tecnologia e non in quelli del problema: la funzione `fetch` per il dialogo con il backend_G, la libreria di notifica per i messaggi all'utente. Se tali interfacce penetrassero nei service, ogni collaudo della logica applicativa richiederebbe un backend_G in esecuzione o un'interfaccia grafica montata, e la verifica dei casi di errore diventerebbe impraticabile.

- **Soluzione:** definire un'interfaccia stabile espressa nei termini del dominio applicativo e realizzarla con una classe che ne traduce le chiamate nell'interfaccia, incompatibile, del meccanismo sottostante.
- **Applicazione nel progetto:** il pattern è applicato a due confini tecnici del sistema.
 - `FetchApiClient` realizza l'interfaccia `ApiClientService`, che dichiara le operazioni `get`, `post`, `put` e `delete` tipizzate. Il metodo privato `request()` concentra la costruzione dell'indirizzo, la serializzazione del corpo e la traduzione degli esiti: un fallimento di rete e una risposta non riuscita diventano entrambi un `ApiError`, che espone il messaggio e lo stato numerico quando disponibile.
 - `NotificationManager` realizza l'interfaccia `NotificationService`, delegando alla libreria *react-hot-toast* la gestione di coda, timer di scomparsa automatica e impilamento dei messaggi.
- **Conseguenze:** I test di `FetchApiClient` verificano il trattamento delle risposte di errore senza alcun backend_G in esecuzione. Da questo ne consegue che ogni comunicazione con il backend_G deve transitare per `ApiClientService`. L'esportazione del `decision treeG`, che allo stato attuale invoca `fetch` direttamente, dovrà essere ricondotta a tale regola.

3.6.3.2. Repository

- **Problema:** il catalogo dei `decision treeG` è l'unico insieme di dati che il backend_G possiede e mette a disposizione dell'utente, e vi_G accede oggi leggendo file JSON dal filesystem. Se i service invocassero direttamente i moduli di accesso al filesystem, il criterio di reperimento dei dati risulterebbe disseminato in ogni punto che ne ha bisogno e la logica applicativa sarebbe collaudabile soltanto predisponendo file reali su disco a ogni esecuzione della suite, rendendo di fatto impraticabile la verifica dei casi di errore.
- **Soluzione:** interporre fra Application Layer e sorgente dei dati un'interfaccia che espone le operazioni sulla collezione di entità come se questa fosse disponibile in memoria, nascondendo al chiamante il meccanismo di accesso effettivo.
- **Applicazione nel progetto:** l'interfaccia astratta `IDecisionTreeRepository` dichiara i metodi `get()`, `save()`, `delete()` e `list()`; l'implementazione concreta `JsonDecisionTreeRepository` li realizza leggendo e scrivendo i file contenuti in `backendG/data/decision_trees/`. Il `DecisionTreeService` riceve l'interfaccia nel costruttore e ignora completamente la natura della sorgente, tanto nel reperimento del singolo albero quanto nelle operazioni che percorrono l'intero catalogo, come la derivazione dei requisiti_G applicabili a un tipo di asset_G. Coerentemente con il principio selettivo enunciato in

, questa è l'unica coppia interfaccia/implementazione del backend_G: `Device` e `AssetG` non dispongono di un repository, non essendo collezioni possedute e servite dal server ma dati che l'utente porta al sistema e riporta via.

- **Conseguenze:** il beneficio è verificabile nel codice esistente, dove i test del `DecisionTreeService` sostituiscono l'implementazione concreta con la classe `FakeDecisionTreeRepository`, che realizza la medesima interfaccia mantenendo gli alberi in memoria e consente quindi di collaudare la logica applicativa senza alcun accesso al disco. L'interfaccia non è motivata dalla previsione di un archivio centralizzato — ipotesi esclusa dalla decisione `VEG-7.1` — bensì da questa verificabilità e dalla necessità, prevista dai casi d'uso di modifica del decision tree_G, di scrivere gli alberi su disco attraverso un unico punto di accesso.

3.6.3.3. Facade

- **Problema:** diverse operazioni che l'utente percepisce come unitarie sono in realtà sequenze articolate. L'importazione di un dispositivo da file richiede il riconoscimento del formato, la lettura asincrona tramite `FileReader`, l'interpretazione del contenuto, la validazione dello schema e infine l'invio al backend_G dei metadati del device e di ciascun asset_G. Esporre tale sequenza alle viste le legherebbe a dettagli estranei alla presentazione e ne impedirebbe il riuso da parte di viste diverse.
- **Soluzione:** raggruppare la sequenza dietro un'unica operazione di alto livello, espressa nel linguaggio del caso d'uso, che coordini internamente i collaboratori necessari.
- **Applicazione nel progetto:** lato client, `importDeviceFromFile(file)` racchiude l'intera sequenza di importazione e restituisce alla vista il solo esito. I custom hook assolvono la funzione analoga verso il Presentation Layer: `useSessionRunner()` coordina le fasi della valutazione guidata, il caricamento e l'idratazione dell'albero e la registrazione dell'esito, esponendo alla pagina soltanto lo stato e le azioni necessarie; `useResult()` e `useSessionModify()` operano allo stesso modo per la consultazione degli esiti e per la ripresa dei requisiti_G. Lato server, `DecisionTreeService.get_tree()` racchiude in una sola chiamata l'accesso al catalogo, la gestione dell'assenza del dato (`DecisionTreeNotFoundError`) e la normalizzazione dell'albero, mentre `create_device()` e `create_asset()` costituiscono il punto unico di validazione delle rispettive entità, condiviso dal percorso di creazione manuale e da quello di importazione.
- **Conseguenze:** pagine e rotte restano prive di logica applicativa e si limitano, rispettivamente, a renderizzare e a deserializzare, delegare e serializzare. L'unicità del punto di validazione evita che due percorsi diversi applichino allo stesso concetto regole divergenti.

3.6.3.4. Factory Method

- **Problema:** un'istanza di Flask creata come oggetto globale a livello di modulo verrebbe condivisa da tutti i test della suite, con il rischio concreto che configurazione e stato residuo si propaghino da un test al successivo, producendo esiti dipendenti dall'ordine di esecuzione. Un problema affine riguarda la costruzione delle entità di dominio a partire dal contenuto dei file di catalogo, dove la classe da istanziare dipende dal dato ricevuto.

- **Soluzione:** delegare a una funzione dedicata la costruzione dell'oggetto, invocabile più volte per ottenere istanze indipendenti, e concentrare in essa la decisione sulla classe concreta da istanziare.
- **Applicazione nel progetto:** `create_app()` costruisce e configura una nuova istanza di Flask, registrando i blueprint e i collaboratori. `create_decision_tree_blueprint(service)` e `create_assets_blueprint(service)` costruiscono i rispettivi blueprint a partire dal service ricevuto: la factory è impiegata esattamente dove un collaboratore deve essere iniettato, mentre il blueprint dei device, le cui rotte invocano direttamente le funzioni di validazione e non dipendono da alcun collaboratore costruito a runtime, è registrato senza factory. Sul versante del dominio, `normalize_tree()` costruisce l'entità `DecisionTree` a partire dal contenuto grezzo del file e delega a `_normalize_node()` la scelta fra `QuestionNode` e `LeafNode` in base al campo `type` del nodo_G.
- **Conseguenze:** ogni test dispone di un'istanza isolata dell'applicazione; `create_app()` è l'unico punto del backend_G in cui compaiono i nomi delle classi concrete, cosicché la direzione delle dipendenze è verificabile ispezionando un solo file; l'eventuale introduzione di una nuova tipologia di nodo_G, richiesta da un'evoluzione della norma, riguarderebbe la sola funzione `_normalize_node()`.

3.6.4. Observer

- **Problema:** lo stato condiviso fra più viste — il dispositivo in lavorazione, la sessione di valutazione, l'albero corrente — deve provocare l'aggiornamento dei soli componenti effettivamente interessati, senza che i moduli che modificano lo stato debbano conoscere i componenti che lo consumano.
- **Soluzione:** i consumatori si registrano presso il detentore dello stato, che notifica automaticamente ogni variazione ai soli osservatori interessati.
- **Applicazione nel progetto:** gli store `Zustand DeviceStore`, `SessionStore` e `TreeStore` costituiscono i soggetti osservati; i componenti si registrano tramite hook selettori, come `useSessionStore((state) => state.session)`, che circoscrivono la sottoscrizione alla sola porzione di stato utilizzata. Le modifiche avvengono unicamente attraverso le azioni tipizzate esposte da ciascuno store: `setDevice`, `updateDeviceDetails`, `addAsset`, `updateAsset` e `removeAsset` per `DeviceStore`; `start`, `ensureSession`, `resume`, `syncProgress`, `completeCurrent`, `select` e `reopen` per `SessionStore`; `loadTree`, `hydrate`, `answer`, `goBack` e `goForward` per `TreeStore`. Ciascuno store espone inoltre `reset`, che ne ripristina lo stato iniziale.
- **Conseguenze:** il flusso dei dati resta unidirezionale e ogni variazione di stato è riconducibile a un'azione esplicita e tracciabile. La suddivisione per area funzionale, in luogo di un unico store globale, estende allo stato condiviso la separazione delle responsabilità adottata a livello di moduli, mentre la granularità dei selettori evita i re-render indiscriminati. Gli store non sono del tutto indipendenti: `DeviceStore` invoca la

reimpostazione di `SessionStore` quando il dispositivo viene sostituito o modificato, poiché una sessione di valutazione riferita a un dispositivo che non esiste più produrrebbe esiti privi di significato. Si tratta di una dipendenza deliberata e a senso unico, che realizza un vincolo di dominio anziché un accoppiamento accidentale.

3.6.4.1. Proxy

- **Problema:** il pattern risponde nel prodotto a due esigenze distinte, entrambe riconducibili al controllo dell'accesso a una risorsa. Da un lato l'albero decisionale di un requisito viene richiesto ripetutamente nel corso di una valutazione, mentre si tratta di dati di catalogo immutabili per l'intera durata della sessione. Dall'altro alcune pagine presuppongono l'esistenza di una sessione attiva e non possono essere raggiunte direttamente per indirizzo, pena l'accesso a una vista priva dei dati che le danno senso.
- **Soluzione:** interporre fra il chiamante e la risorsa un sostituto che ne espone la medesima interfaccia e ne governa l'accesso, aggiungendovi la conservazione del risultato oppure la verifica delle precondizioni.
- **Applicazione nel progetto:** nella variante con conservazione, `DecisionTreeService` avvolge le proprie chiamate in `queryClient.fetchQuery()`, che restituisce il risultato già ottenuto per la medesima chiave anziché ripetere la richiesta. La configurazione dichiara `staleTime` e `gcTime` illimitati e disabilita i ritentativi, coerentemente con la natura immutabile del dato: il proxy serve a non rileggere più volte lo stesso catalogo, non a compensare l'inaffidabilità della comunicazione. L'accesso alla rete continua ad avvenire tramite l'interfaccia `ApiClientService`, che il service riceve nel costruttore. Nella variante con verifica delle precondizioni, il componente `RequireSession` avvolge le pagine di esecuzione e di modifica della sessione: presenta al router la stessa interfaccia della pagina protetta e ne consente il rendering solo in presenza di una sessione, reindirizzando altrimenti alla pagina iniziale.
- **Conseguenze:** i service applicativi non contengono logica di conservazione dei risultati e le pagine non contengono controlli di accesso; entrambi i sostituti sono rimovibili senza modificare il codice che protegge. Si segnala che la conservazione dei risultati è utilizzata in modo imperativo e non attraverso gli hook della libreria: l'applicazione non monta alcun provider, e il proxy resta pertanto confinato all'Infrastructure Layer.

3.6.4.2. Strategy

- **Problema:** il dispositivo e i suoi `assetG` devono poter essere scritti e letti in formati diversi, oggi JSON e CSV. La decisione `VEG-7.1` accresce il peso di questa funzionalità: poiché i `decision treeG` modificati non si sincronizzano fra le installazioni ma vengono esportati e distribuiti manualmente, i file di scambio sono il canale con cui gli utenti si trasmettono il lavoro svolto. Governare la varietà dei formati con una struttura condizionale interna al modulo di importazione ed esportazione comporterebbe di modificarlo a ogni nuova esigenza, con conseguente rischio di regressione sui formati già funzionanti.

- **Soluzione:** definire una famiglia di algoritmi intercambiabili dietro un'interfaccia comune e selezionare a runtime l'implementazione corrispondente al formato richiesto.
- **Applicazione nel progetto:** l'interfaccia `DeviceFileFormat` dichiara l'estensione, il tipo di contenuto e le due operazioni simmetriche `serialize(device)` e `parse(text)`; le implementazioni concrete `jsonDeviceFormat` e `csvDeviceFormat` le realizzano per i rispettivi formati, ciascuna con le proprie regole di interpretazione — nel caso del formato tabellare, l'intestazione attesa, la codifica dei campi contenenti separatori e la rappresentazione degli `assetG` su righe successive. La funzione `formatForFile(file)` seleziona la strategia in base al file fornito dall'utente; `importDeviceFromFile()` e `exportDevice()` la utilizzano senza conoscerne l'implementazione.
- **Conseguenze:** l'aggiunta di un formato richiede la sola introduzione di una nuova implementazione dell'interfaccia, senza modifiche ai moduli chiamanti né alle viste, e ciascuna strategia è verificabile con unit test indipendenti dalle altre.

3.6.5. Tabella riassuntiva

Pattern	Ambito	Moduli coinvolti
Adapter	Frontend _G	ApiClientService/FetchApiClient, NotificationService/NotificationManager
Repository	Backend _G	IDecisionTreeRepository, JsonDecisionTreeRepository, DecisionTreeService
Facade	Frontend _G , Backend _G	importDeviceFromFile(), hook applicativi (useSessionRunner, useResult, useSessionModify), DecisionTreeService, create_device(), create_asset()
Factory Method	Backend _G	create_app(), create_decision_tree_blueprint(), create_assets_blueprint(), normalize_tree(), _normalize_node()
Observer	Frontend _G	DeviceStore, SessionStore, TreeStore e relativi hook selettori
Proxy	Frontend _G	queryClient su DecisionTreeService, componente RequireSession
Strategy	Frontend _G	DeviceFileFormat, jsonDeviceFormat, csvDeviceFormat, formatForFile()

4. Decomposizione del sistema

Questa sezione descrive l'organizzazione interna delle due unità del sistema. Per ciascun livello sono indicati i moduli che lo compongono, la responsabilità di ciascuno e le dipendenze verso gli altri livelli. Le entità di dominio sono trattate separatamente in Sezione 5.

4.1. Diagramma dei package

4.2. Frontend_G

Il frontend_G è realizzato come **Single Page Application** in React. Le pagine non contengono logica di business: compongono l'interfaccia e delegano recupero dei dati, validazioni e transizioni di stato a hook applicativi dedicati, che a loro volta si appoggiano su store condivisi e su service.

4.2.1. Organizzazione a livelli

Il frontend_G è organizzato in cinque livelli:

Livello	Contenuto
Presentation	Pagine e componenti React, instradamento con React Router.
Application	Hook applicativi e service che orchestrano i casi d'uso.
State Management	Store Zustand distribuiti per area funzionale.
Infrastructure	Comunicazione HTTP, memorizzazione dei risultati, notifiche.
Domain	Entità di business, regole pure, schemi di validazione Zod.

La dipendenza fra livelli è a senso unico: Presentation dipende da Application; Application dipende da State, Domain e Infrastructure; Domain non dipende da alcun altro livello. È questo vincolo a rendere sostituibile un dettaglio tecnico senza toccare le pagine.

4.2.2. Presentation Layer

Il livello comprende nove pagine, ciascuna corrispondente all'ingresso di un caso d'uso.

- **HomePage**: creazione di un nuovo dispositivo, importazione da file o ripresa di una sessione salvata.
- **DeviceFormPage**: raccolta dei dati descrittivi del dispositivo, in creazione o in modifica.
- **DeviceAssetManagementPage**: gestione dell'elenco degli asset_G associati al dispositivo.
- **AssetFormPage**: creazione o modifica di un asset_G e assegnazione dei requisiti_G da valutare.
- **DeviceSummaryPage**: riepilogo di dispositivo e asset_G prima dell'avvio della valutazione.
- **SessionRunnerPage**: presentazione della domanda corrente e navigazione fra le risposte.

- **ModifySessionPage**: scelta del requisito da riprendere o rivalutare.
- **ResultPage**: consultazione degli esiti e loro esportazione.
- **DecisionTreeCatalogPage**: elenco dei decision tree_G disponibili, dettaglio, importazione ed esportazione.

I componenti riutilizzabili sono tre:

- **Esito**: etichetta che rappresenta lo stato di valutazione con un codice colore.
- **GrafoDecisionTree**: rappresentazione grafica dell'albero, con evidenziazione del nodo_G corrente e del percorso già intrapreso.
- **RequireSession**: componente di guardia che avvolge le pagine richiedenti una sessione attiva e reindirizza alla pagina iniziale in sua assenza.

L'instradamento è definito in `App.tsx`, che associa a ciascuna vista un percorso e applica `RequireSession` alle rotte `/session` e `/session/modify`.

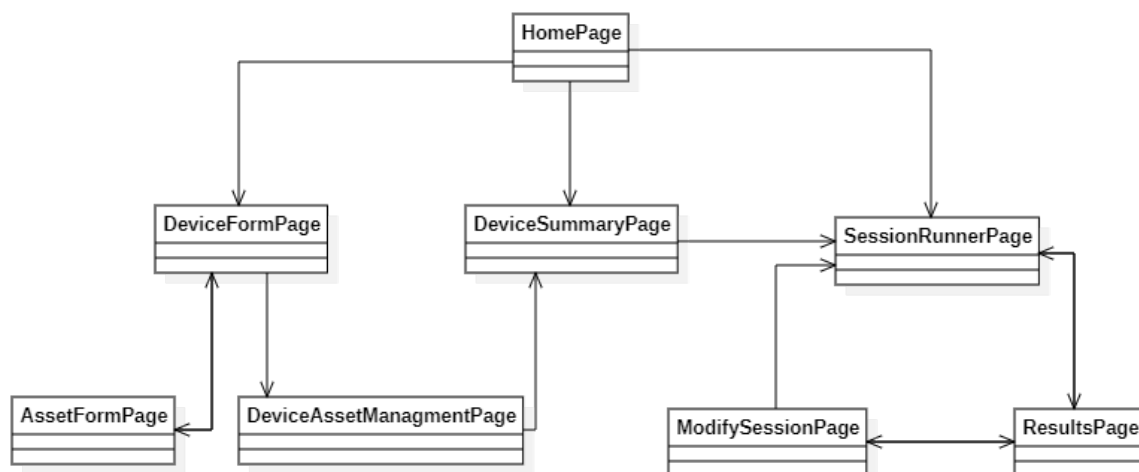


Figura 1: Diagramma delle pagine

4.2.3. Application Layer

La logica operativa risiede in due famiglie di moduli: gli **hook**, al servizio di una sola pagina, e i **service**, che raccolgono operazioni utili in più punti.

Gli hook espongono alla pagina il solo stato da mostrare e le azioni da collegare ai comandi, trattenendo la sequenza di passaggi che li produce.

- **useSessionRunner**: conduce la valutazione guidata, dalla selezione dell'asset_G e del requisito fino alla registrazione dell'esito; carica l'albero al momento opportuno e riporta il percorso al punto raggiunto quando una sessione viene ripresa.

- **useResult**: gestisce la consultazione degli esiti, consentendo di scendere dall'asset_G al singolo requisito e ricostruendo la sequenza di domande e risposte che ha condotto al risultato.
- **useSessionModify**: prepara la ripresa o la rivalutazione di un requisito e determina quali altri requisiti_G ne dipendono, poiché rifacendolo vanno rifatti anche quelli.

I service raccolgono le operazioni indipendenti dalla singola pagina.

- **DeviceService**: creazione manuale, importazione da file, creazione e modifica di un asset_G, esportazione del dispositivo. È realizzato come insieme di funzioni esportate anziché come classe.
- **DecisionTreeService**: recupero dell'elenco del catalogo e del singolo albero, importazione ed esportazione. È l'unico service realizzato come classe e riceve nel costruttore l'implementazione di `ApiClientService`, così da poter essere collaudato con un doppio di test.
- **SessionService**: produzione e riletture validate del file di sessione, e ri_G-esportazione delle regole di costruzione del piano di valutazione.
- **deviceFileFormats**: raccoglie i formati con cui il dispositivo può essere scritto e letto, oggi JSON e CSV, ciascuno con le proprie regole di serializzazione e interpretazione.

4.2.4. State Management

Lo stato condiviso fra più pagine è distribuito su tre store Zustand indipendenti.

Store	Stato	Azioni
DeviceStore	Dispositivo in lavorazione e relativo payload.	setDevice, updateDeviceDetails, addAsset, updateAsset, removeAsset, reset
SessionStore	Sessione di valutazione: coppie asset _G -requisito, coppia corrente, percorsi ed esiti.	start, ensureSession, resume, syncProgress, completeCurrent, select, reopen, reset
TreeStore	Albero del requisito corrente, nodo _G attuale, cronologia delle risposte e cursore.	loadTree, hydrate, answer, goBack, goForward, reset

Ogni componente dichiara tramite un selettore la porzione di stato che gli occorre e viene aggiornato solo quando quella porzione cambia. Le modifiche passano esclusivamente da azioni dichiarate, mai da scritture dirette.

I tre store non sono del tutto indipendenti: quando il dispositivo viene sostituito o modificato, DeviceStore azzerava SessionStore. Si tratta di una dipendenza deliberata e a

senso unico, che realizza un vincolo di dominio — una valutazione riferita a un dispositivo nel frattempo mutato produrrebbe esiti privi di significato.

Il cursore di `TreeStore` merita una precisazione: `goBack` e `goForward` spostano la posizione lungo la cronologia senza scartare le risposte successive, che vengono invalidate solo quando l'utente risponde diversamente a un nodo_G già risposto.

4.2.5. Infrastructure Layer

- **ApiClientService** e **FetchApiClient**: l'interfaccia con cui il resto del codice richiede dati al backend_G e la sua realizzazione concreta su `fetch`. Il metodo privato `request()` concentra la composizione dell'indirizzo, la serializzazione del corpo e la traduzione degli esiti: tanto un'assenza di rete quanto una risposta non riuscita diventano un `ApiError`, che espone il messaggio e, quando disponibile, il codice di stato.
- **queryClient**: istanza di `TanStack Query` impiegata in modo imperativo per conservare gli alberi già richiesti. La configurazione disabilita scadenza e ritentativi, coerentemente con la natura immutabile del dato durante una sessione.
- **NotificationService** e **NotificationManager**: l'interfaccia per avvisare l'utente e la sua realizzazione su `react-hot-toast`.

Grazie a questa separazione, sostituire il modo in cui si dialoga con il backend_G o si presentano gli avvisi non tocca né le pagine né i service.

4.2.6. Domain Layer

Il livello rappresenta il problema applicativo in modo indipendente da React, dal routing e dal backend_G. Comprende le entità — descritte in scritte come funzioni pure, e quindi verificabili in isolamento.

- **treeRules**: percorre l'albero. `resolveNodeId()` ricava il nodo_G corrente ripercorrendo le risposte dalla radice, `currentOutcome()` e `isRequirementComplete()` riconoscono il raggiungimento di una foglia, `describePath()` ricostruisce la sequenza di domande e risposte da presentare nei risultati.
- **sessionRules**: governa la valutazione. `buildPlan()` costruisce l'elenco delle coppie asset_G-requisito, `getEvaluationProgress()` calcola l'avanzamento, `transitiveDependents()` determina quali requisiti_G dipendono da quello in corso di rivalutazione. Le funzioni `getEvaluationStatus()`, `getAssetStatus()` e `getDeviceStatus()` riassumono più esiti in uno solo secondo una priorità fissa che fa prevalere il fallimento e la valutazione in corso sugli esiti positivi.
- **treeLayout**: dispone i nodi in colonne e livelli per il disegno del grafo, producendo coordinate astratte e non pixel, così da restare indipendente dalla tecnologia di rendering.

La validazione dei dati in ingresso — input dell'utente, risposte del backend_G, contenuto dei file importati — è affidata a schemi **Zod** definiti accanto alle entità.

4.2.7. Flussi applicativi

Il flusso dei dati è unidirezionale. Le pagine non modificano lo stato condiviso né dialogano con il backend_G; raccolgono l'azione dell'utente e la inoltrano all'hook, che decide come procedere. Se occorre un dato che il client non possiede, l'hook lo richiede al service, che passa dall'Infrastructure Layer; altrimenti agisce direttamente sugli store.

I flussi principali sono i seguenti.

- **Nuova valutazione:** da HomePage si passa a DeviceFormPage per i dati del dispositivo, quindi a DeviceAssetManagementPage e AssetFormPage per gli asset_G, infine a DeviceSummaryPage per il riepilogo. Il dispositivo resta nel DeviceStore per tutto il percorso.
- **Importazione di un dispositivo:** il formato è riconosciuto dall'estensione tramite `formatForFile()`, il contenuto è letto e validato con Zod, quindi i metadati del dispositivo e ciascun asset_G sono inviati al backend_G, che assegna gli identificativi mancanti e deriva i requisiti_G applicabili.
- **Avvio della valutazione:** da DeviceSummaryPage si entra in SessionRunnerPage; la sessione è creata a partire dal piano prodotto da `buildPlan()`. Se una sessione compatibile è già in corso, `ensureSession()` la riprende anziché ricominciare.
- **Esecuzione del decision tree_G:** scelto il requisito, l'albero è caricato e il percorso riportato al punto raggiunto tramite `hydrate()`. Ogni risposta aggiorna il percorso nel client e allinea la sessione, cosicché un salvataggio colga sempre lo stato corrente. Raggiunta una foglia, l'esito è registrato con `completeCurrent()`.
- **Ripresa e modifica:** `ModifySessionPage` consente di riaprire un requisito già chiuso; insieme a esso `reopen()` riapre i requisiti_G che ne dipendono, il cui esito potrebbe cambiare.
- **Consultazione degli esiti:** valutate tutte le coppie, `ResultPage` consente di scendere dall'asset_G al requisito e rivedere domande e risposte.
- **Salvataggio e ripresa da file:** la sessione può essere scaricata in qualsiasi momento e ricaricata da HomePage, previa validazione dello schema.
- **Esportazione:** il dispositivo è scritto in JSON o CSV dal client; il decision tree_G è prodotto dal backend_G, che risponde all'endpoint di esportazione nel formato richiesto.

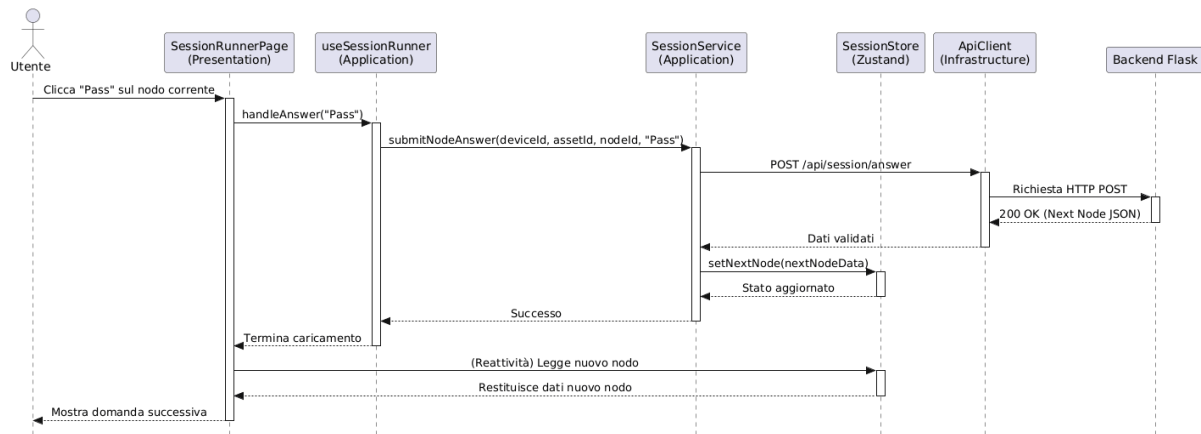


Figura 2: Diagramma di sequenza dell'avanzamento nell'albero decisionale

4.3. Backend_G

Il backend_G è un'applicazione Flask esposta come singola API REST stateless, organizzata in quattro livelli.

4.3.1. Organizzazione a livelli

Livello	Contenuto
Presentation	Blueprint Flask che espongono gli endpoint REST.
Application	Service che orchestrano i casi d'uso.
Domain	Entità e loro regole, senza dipendenze da Flask o dal filesystem.
Persistence	Interfaccia e implementazione di accesso al catalogo.

Presentation dipende da Application, Application da Domain e Persistence, Domain da nulla. La direzione è verificata dai test di dominio, che non richiedono alcun contesto applicativo Flask.

4.3.2. Presentation Layer

Il livello raggruppa quattro blueprint. Ogni rotta riceve la richiesta, delega l'elaborazione al service competente e traduce l'esito in una risposta HTTP con il codice di stato appropriato, senza contenere logica di business.

L'applicazione è composta da `create_app()`, che costruisce il repository, lo inietta nel `DecisionTreeService` e passa quest'ultimo alle factory dei blueprint che ne hanno bisogno. Il blueprint dei dispositivi, le cui rotte invocano direttamente la funzione di validazione e non dipendono da collaboratori costruiti a runtime, è registrato senza factory.

4.3.3. Application Layer

- **create_device()**: valida i metadati del dispositivo (nome, sistema operativo, descrizione) e ne costruisce l'entità, rispettando l'id se fornito o generandone uno con uuid4. È il punto unico di validazione, condiviso dalla creazione manuale e dall'importazione.

- **create_asset()**: valida i campi dell'asset_G e, quando requirements non è fornito, deriva i requisiti_G applicabili interrogando `DecisionTreeService.list_requirement_ids_for_type()`.
- **DecisionTreeService**: unica classe del livello. Carica un albero tramite il repository, ne verifica l'integrità e lo normalizza; espone inoltre `list_trees()`, `list_requirement_ids_for_type()` e `import_tree()`.

Le prime due sono realizzate come funzioni esportate anziché come classi, non avendo stato né collaboratori da conservare fra le invocazioni.

4.3.4. Domain Layer

Il livello comprende le entità Device, Asset_G, Node — con le specializzazioni QuestionNode e LeafNode — DecisionTree e Session, tutte realizzate come dataclass immutabili prive di dipendenze da librerie web o di accesso ai file. Le operazioni di navigazione sono definite come metodi delle entità: DecisionTree.get_node() reperisce un nodo_G per identificatore, Node.next() restituisce il successore per il ramo scelto, Node.verdict() l'esito di una foglia.

Session e le strutture correlate (PathStep, Evaluation, Current) sono definite lato server per rappresentare il file di sessione, ma non sono esposte da alcuna rotta: la sessione è gestita interamente dal client.

4.3.5. Persistence Layer

Il livello isola l'accesso ai dati dietro l'interfaccia IDecisionTreeRepository, realizzata da JsonDecisionTreeRepository sui file in backend_G/data/decision_trees/. Il principio selettivo che governa l'esistenza di un solo repository è discusso in Sezione 3.

4.3.6. Endpoint REST

Metodo	Percorso	Descrizione
GET	/health	Verifica di disponibilità del servizio.
POST	/devices	Valida i metadati del dispositivo, risolve l'id e restituisce l'entità serializzata con assets sempre vuoto. Nessuna scrittura su disco. Contratto condiviso da creazione manuale e importazione.
POST	/assets	Valida i campi dell'asset _G e ne deriva i requisiti _G applicabili quando non forniti, interrogando il catalogo. Risolve l'id. Nessuna scrittura su disco.
GET	/decision-trees	Restituisce l'elenco sintetico dei decision tree _G disponibili.
GET	/decision-trees/{requirementId}	Carica il decision tree _G indicato, ne verifica l'integrità e lo restituisce normalizzato. Risponde 404 se assente.

POST	/decision-trees/import	Riceve un file JSON o CSV come multipart/form-data, ne valida la struttura e lo registra nel catalogo tramite il repository. Risponde 400 in caso di struttura non valida.
GET	/decision-trees/{requirementId}/export	Restituisce il decision tree _G nel formato indicato dal parametro format, json o csv.

4.3.7. Testing

I test del backend_G risiedono in una cartella `tests/` separata, a specchio di `src/`, secondo la convenzione idiomatica di Pytest; il frontend_G adotta invece la convenzione idiomatica di Vitest, con i test collocati accanto al file verificato. Il Domain Layer è verificato con unit test; i service con test che sostituiscono il repository con `FakeDecisionTreeRepository`; le rotte con test di integrazione tramite il client di test di Flask. La qualità del codice è verificata in integrazione continua tramite `ruff` lato backend_G ed `eslint` lato frontend_G.

5. Diagrammi delle classi

5.1. Criteri di rappresentazione

Questa sezione descrive la struttura statica del dominio applicativo, cioè le entità che rappresentano i concetti di business su cui si fonda la valutazione di conformità EN 18031 e le relazioni che le legano. I moduli che orchestrano i casi d'uso — pagine, hook, service, store, rotte — sono trattati in Sezione 4 e non compaiono qui.

Il dominio esiste in due realizzazioni distinte, una per ciascuna unità del sistema. Non si tratta di una duplicazione accidentale: le due forme rispondono a esigenze diverse e presentano differenze deliberate, discusse in Sezione 5.4.

Nelle schede che seguono sono riportati, per ciascuna entità, il ruolo, gli attributi con il rispettivo tipo, i metodi che portano comportamento e la responsabilità complessiva. Sono deliberatamente omessi i metodi di accesso ai singoli campi, che nel frontend_G sono *getter* e nel backend_G proprietà di sola lettura: elencarli non aggiungerebbe informazione alla struttura. La corrispondenza fra entità e requisiti_G è riportata in

5.2. Dominio frontend_G

Tutte le entità del dominio frontend_G sono **immutabili**: i campi sono dichiarati privati tramite la sintassi dei campi privati di JavaScript (`#campo`) e le operazioni di modifica non alterano l'istanza, ma restituiscono una nuova istanza con il valore aggiornato. Da qui la convenzione di denominazione `with*` adottata per tali metodi. L'immutabilità garantisce che nessun modulo possa alterare uno stato condiviso in modo non tracciabile e si combina con la propagazione delle modifiche degli store, che rilevano il cambiamento per identità del riferimento.

Ciascuna entità è affiancata da uno o più schemi **Zod** che ne descrivono la forma attesa e dai quali sono derivati i tipi TypeScript corrispondenti. Il metodo statico `create()` costituisce

l'unico punto di costruzione a partire da dati esterni: valida il dato grezzo contro lo schema e costruisce l'istanza solo se la validazione ha successo.

5.2.1. Device

Ruolo: rappresenta il dispositivo sottoposto a valutazione e aggrega gli `assetG` che lo compongono.

Attributi

- `id`: `string` — identificatore univoco, assegnato dal `backendG` o presente nel file importato.
- `name`: `string` — denominazione del dispositivo.
- `operatingSystem`: `string` — sistema operativo del dispositivo.
- `description`: `string` — descrizione testuale.
- `assets`: `AssetG[]` — gli `assetG` associati, in composizione.

Metodi

- `withDetails(patch)`: `Device` — restituisce un dispositivo con i dati descrittivi aggiornati.
- `withAssets(assets)`: `Device`, `withAssetAdded(assetG)`: `Device`, `withAssetUpdated(assetG)`: `Device`, `withAssetRemoved(assetId)`: `Device` — restituiscono un dispositivo con l'elenco degli `assetG` modificato.
- `buildPlan()`: `{ assetId, requirementId }[]` — costruisce l'elenco delle coppie `assetG-requisito` da valutare, percorrendo gli `assetG` e i `requisitiG` assegnati a ciascuno.
- `toJSON()`: `object` — produce la rappresentazione serializzabile, usata dall'esportazione e dal file di sessione.
- `create(raw)`: `Device` — costruisce l'entità validando il dato grezzo con `DeviceSchema`.

Responsabilità: raccogliere i dati descrittivi del dispositivo, custodire l'elenco degli `assetG` e derivare il piano di valutazione che costituisce il punto di partenza di ogni sessione.

Schemi associati: `DeviceSchema` (forma completa), `DeviceCreateSchema` (dati raccolti nel form di creazione), `DeviceImportSchema` (forma attesa in un file importato, con `id` facoltativo e `assetG` annidati).

5.2.2. Asset_G

Ruolo: rappresenta un elemento del dispositivo soggetto a valutazione, al quale sono associati i `requisitiG` EN 18031 applicabili.

Attributi

- `id`: `string` — identificatore univoco all'interno del dispositivo.
- `name`: `string` — denominazione dell'`assetG`.
- `type`: `AssetType` — categoria funzionale, ristretta ai valori `network`, `security`, `privacy` e `financial`.
- `description`: `string` — descrizione testuale.
- `sensitive`: `boolean` — indica se l'`assetG` tratta dati sensibili.
- `requirements?`: `string[]` — codici dei requisiti_G assegnati, derivati dal `backendG` in base al tipo.

Metodi

- `withDetails(patch)`: `AssetG` — restituisce un `assetG` con i dati descrittivi aggiornati, conservando i requisiti_G assegnati.
- `toJSON()`: `object` — produce la rappresentazione serializzabile.
- `create(raw)`: `AssetG` — costruisce l'entità validando il dato grezzo con `AssetSchema`.

Responsabilità: custodire i metadati dell'`assetG` e i requisiti_G a esso applicabili.

L'`assetG` non conserva il proprio stato di valutazione: quello stato appartiene alla sessione ed è calcolato da `getAssetStatus()` a partire dalle valutazioni registrate. La scelta evita che la stessa informazione esista in due luoghi con il rischio di divergere.

5.2.3. DecisionTree

Ruolo: rappresenta l'albero decisionale che guida la valutazione di un requisito.

Attributi

- `requirementId`: `string` — codice del requisito, conforme all'espressione regolare che ne descrive il formato.
- `requirementName`: `string` — denominazione estesa del requisito.
- `version?`: `string` — versione dell'albero, nel formato a tre numeri.
- `appliesTo?`: `string[]` — tipi di `assetG` ai quali il requisito si applica; è il campo su cui il `backendG` deriva i requisiti_G applicabili.
- `dependencies?`: `string[]` — requisiti_G dai quali questo dipende.
- `rootNode`: `string` — identificatore del nodo_G radice.
- `nodes`: `Node[]` — collezione dei nodi in forma piatta.
- `message?`: `string` — messaggio restituito dal `backendG` a seguito di un'importazione.

Metodi

- `getNode(id): Node` — restituisce il nodo_G corrispondente all'identificatore, sollevando un errore se assente.
- `toJSON(): object`, `create(raw): DecisionTree` — come per le altre entità.

Responsabilità: rappresentare la struttura dell'albero e consentirne l'accesso per identificatore, operazione richiesta a ogni passo della navigazione.

I nodi sono conservati in una **lista piatta** e i collegamenti sono espressi per identificatore, anziché in una struttura di contenimento ricorsiva. La ragione decisiva è che rami distinti devono poter convergere sullo stesso nodo_G, situazione presente negli alberi della norma: una struttura di contenimento richiederebbe di duplicare il nodo_G condiviso in ciascun ramo che lo raggiunge, con il rischio che le copie divergano. La forma piatta coincide inoltre con quella dei file di catalogo, evitando una conversione strutturale a ogni serializzazione.

5.2.4. Node, QuestionNode e LeafNode

Ruolo: elemento atomico dell'albero. Un nodo_G è una domanda con due rami oppure una foglia che porta un esito.

Il dominio frontend_G non adotta una gerarchia di ereditarietà: `QuestionNode` e `LeafNode` sono classi indipendenti che realizzano il medesimo contratto `NodeContract`, e il tipo `Node` è definito come loro **unione discriminata** sul campo `type`. La scelta è motivata dal sistema dei tipi: l'unione discriminata consente al compilatore di restringere il tipo in base al valore di `type`, cosicché una verifica come `node.type === "leaf"` renda accessibile il campo `outcome` senza conversioni esplicite. Una classe base astratta con `extends` non offrirebbe questa proprietà.

QuestionNode

- `id: string`, `text: string`, `branches: { yes: string, no: string }`.
- `next(answer: boolean): string` — restituisce l'identificatore del nodo_G successivo per il ramo scelto.
- `verdict(): null` — un nodo_G domanda non porta esito.

LeafNode

- `id: string`, `outcome: Outcome`, `text?: string`, dove `Outcome` è ristretto a `PASS`, `FAIL` e `NOT_APPLICABLE`.
- `next(): string` — solleva un errore, non esistendo un successore.
- `verdict(): Outcome` — restituisce l'esito assegnato.

La funzione `createNode(raw)` sceglie la classe concreta da istanziare in base al campo `type` del dato grezzo.

5.2.5. Session

Ruolo: rappresenta la valutazione nel suo insieme: quali coppie asset_G-requisito devono essere esaminate, quale è in corso, quale percorso è stato seguito su ciascuna e con quale esito.

Attributi

- `id`: `string` — identificatore della sessione.
- `savedAt`: `string` — istante dell'ultimo salvataggio.
- `status`: `"in_progress" | "completed"` — stato complessivo.
- `device`: `Device` — il dispositivo valutato, incorporato per intero nella sessione.
- `decisionTreeVersions?`: `Record<string, string>` — versione dell'albero impiegata per ciascun requisito.
- `current?`: `Current` — coppia asset_G-requisito in corso e nodo_G raggiunto.
- `evaluations`: `Evaluation[]` — una voce per ciascuna coppia del piano.

Metodi

- `selectEvaluation(assetId, requirementId)`: `Session` — attiva una coppia e apre l'albero dall'inizio (UC-19).
- `reopenEvaluation(assetId, requirementId, dependents)`: `Session` — riporta a «non valutato» la coppia indicata e quelle che ne dipendono transitivamente (UC-27.2).
- `syncProgress(nodeId, path)`: `Session` — registra nodo_G corrente e percorso parziale senza chiudere la valutazione.
- `completeCurrent(outcome, path)`: `Session` — registra l'esito raggiunto; la sessione passa a «completata» quando tutte le valutazioni lo sono (UC-23).
- `matchesPlan(device)`: `boolean` — verifica che le valutazioni coprano esattamente il piano attuale del dispositivo, condizione che rende la sessione riprendibile così com'è (UC-26).
- `withSavedAt()`, `withEvaluations()`, `withDevice()` — restituiscono una sessione con il rispettivo campo aggiornato.

Responsabilità: custodire lo stato complessivo della valutazione e le transizioni ammesse fra i suoi stati, garantendo che ogni modifica produca una sessione coerente.

La sessione **incorpora** il dispositivo anziché referenziarlo per identificatore: il file di sessione è così autosufficiente e può essere ripreso senza disporre altrove del dispositivo. `matchesPlan()` verifica al momento della ripresa che il dispositivo incorporato produca lo stesso piano, condizione senza la quale la sessione non sarebbe più significativa.

Le strutture `Evaluation`, `Current` e `PathStep` restano dati puri, definiti come tipi derivati dai rispettivi schemi e non come classi: non possiedono comportamento proprio né invarianti da proteggere oltre alla forma, che è già garantita dalla validazione.

5.3. Dominio `backendG`

Le entità del `backendG` sono realizzate come `dataclass` immutabili, con campi privati esposti tramite proprietà di sola lettura. La costruzione a partire da dati esterni avviene tramite i metodi di classe `create()`, che validano i campi obbligatori e sollevano `InvalidDeviceDataError` o `InvalidAssetDataError` in caso di dato non conforme.

- **Device** — `id`, `name`, `operating_system`, `description`, `assets`. Il metodo `create(data)` verifica la presenza dei campi obbligatori e risolve l'identificatore, rispettandolo se fornito o generandolo con `uuid4` altrimenti.
- **Asset_G** — `id`, `name`, `type`, `description`, `sensitive`, `requirements`. Il tipo è rappresentato dall'enumerazione `AssetType`, che espone `from_string()` per la conversione dal valore testuale ricevuto.
- **DecisionTree** — `requirement_id`, `requirement_name`, `root_node`, `nodes`, `version`, `applies_to`, `dependencies`. Il metodo `get_node(node_id)` reperisce un nodo_G per identificatore.
- **Node** — classe astratta che dichiara la proprietà `id` e i metodi `next(answer)` e `verdict()`. È specializzata da `QuestionNode`, che espone `text` e `branches` e realizza la navigazione, e da `LeafNode`, che espone `outcome` e restituisce l'esito. L'enumerazione `NodeOutcome` rappresenta i tre esiti ammessi, la `dataclass` `Branches` la coppia di rami.
- **Session**, con le strutture correlate `Evaluation`, `Current` e `PathStep` — rappresenta il file di sessione. Nessuna rotta la espone: è definita per completezza del modello, ma la sessione è gestita interamente dal client.

5.4. Corrispondenza fra i due domini

Le due realizzazioni descrivono i medesimi concetti, ma differiscono in tre punti, ciascuno per una ragione precisa.

Rappresentazione dei nodi. Il `backendG` adotta una gerarchia di ereditarietà a partire da una classe astratta; il `frontendG` un'unione discriminata su un contratto comune. La differenza discende dai rispettivi sistemi dei tipi: il restringimento del tipo su un campo discriminante è una proprietà di TypeScript senza equivalente diretto in Python, dove l'ereditarietà con metodi astratti è la costruzione idiomatica.

Presenza della sessione. `Session` esiste su entrambi i lati, ma solo il `frontendG` la utilizza. La definizione lato server documenta la forma del file di sessione senza che alcuna rotta la impieghi, coerentemente con il criterio di possesso del dato: la sessione appartiene all'utente, non al sistema.

Estensione del comportamento. Le entità del frontend_G espongono più comportamento — costruzione del piano, transizioni della sessione — perché è nel client che risiede la logica di valutazione. Le entità del backend_G si limitano alla validazione in ingresso e alla navigazione dell'albero, che sono le sole responsabilità del server.

Le due realizzazioni non sono generate l'una dall'altra né condividono codice: la coerenza fra esse è garantita dal contratto degli endpoint, che costituisce l'unica fonte di verità condivisa. Il presidio di tale coerenza è discusso in

5.5. Vincoli e regole di integrità

Le seguenti regole definiscono l'integrità del dominio e sono verificate dal backend_G in fase di validazione di un albero, all'atto del caricamento e dell'importazione.

- Ogni nodo_G possiede un identificatore univoco all'interno del medesimo decision tree_G.
- Il nodo_G radice dichiarato deve esistere fra i nodi dell'albero.
- Ogni ramo di un nodo_G domanda deve puntare a un nodo_G esistente: non sono ammessi riferimenti pendenti.
- Un nodo_G domanda deve possedere testo non vuoto ed entrambi i rami; un nodo_G foglia deve possedere un esito fra PASS, FAIL e NOT_APPLICABLE.
- Il codice del requisito deve rispettare il formato previsto.
- L'albero non deve contenere cicli.

Le regole relative alla modifica strutturale di un decision tree_G — divieto di eliminare il nodo_G radice, prevenzione delle dipendenze circolari fra requisiti_G, trasformazione dei rami non collegati in nodi foglia, annullamento delle modifiche — discendono da requisiti_G opzionali non ancora realizzati e saranno documentate contestualmente alla loro implementazione.

6. Tracciamento dei requisiti_G

In questa sezione è riportato lo stato di copertura dei requisiti_G funzionali individuati nell'[Analisi dei Requisiti_G](#), rispetto a quanto realizzato nel prodotto descritto dal presente documento. Per ciascun requisito sono riportati il codice identificativo, la descrizione e lo stato.

Gli stati impiegati sono due:

- **Soddisfatto:** il requisito è realizzato nel prodotto ed è verificabile attraverso l'uso dell'applicazione.
- **Non soddisfatto:** il requisito non è realizzato nella versione corrente del prodotto.

I codici mantengono la classificazione per priorità adottata in sede di analisi: obbligatori (RF-Ob), desiderabili (RF-D) e opzionali (RF-Op). Le sezioni che seguono ne riportano lo stato distintamente per ciascuna categoria.

6.1. Requisiti_G funzionali obbligatori

Costituiscono il nucleo funzionale imprescindibile del prodotto: il loro soddisfacimento è condizione necessaria affinché l'applicazione risponda ai bisogni primari espressi dalla proponente_G.

Codice	Descrizione	Stato
RF-Ob01	Il sistema deve permettere l'inserimento di un nuovo dispositivo all'interno della piattaforma.	Soddisfatto
RF-Ob02	Il sistema deve permettere l'importazione di un dispositivo tramite un file di configurazione in formato JSON o CSV.	Soddisfatto
RF-Ob03	Il sistema deve permettere la selezione del file sorgente per l'importazione del dispositivo.	Soddisfatto
RF-Ob04	Il sistema deve supportare la selezione di un file in formato JSON come sorgente per l'importazione del dispositivo.	Soddisfatto
RF-Ob05	Il sistema deve supportare la selezione di un file in formato CSV come sorgente per l'importazione del dispositivo.	Soddisfatto
RF-Ob06	Il sistema deve verificare la validità strutturale e la conformità del file di configurazione caricato.	Soddisfatto
RF-Ob07	Il sistema deve bloccare l'importazione e mostrare un messaggio di errore se il file ha un formato non valido.	Soddisfatto
RF-Ob08	Il sistema deve permettere la creazione manuale di un nuovo dispositivo.	Soddisfatto
RF-Ob09	Il sistema deve richiedere l'inserimento dei dati identificativi del dispositivo durante la creazione manuale.	Soddisfatto
RF-Ob10	Il sistema deve permettere l'inserimento del nome identificativo del dispositivo.	Soddisfatto

Codice	Descrizione	Stato
RF-Ob11	Il sistema deve permettere l'inserimento del sistema operativo del dispositivo.	Soddisfatto
RF-Ob12	Il sistema deve permettere l'inserimento di una descrizione testuale del dispositivo.	Soddisfatto
RF-Ob13	Il sistema deve validare i dati inseriti nei form e mostrare un errore in caso di campi vuoti o non conformi.	Soddisfatto
RF-Ob14	Il sistema deve permettere la visualizzazione delle informazioni e dei dati relativi al dispositivo.	Soddisfatto
RF-Ob15	Il sistema deve mostrare in dettaglio il nome del dispositivo registrato.	Soddisfatto
RF-Ob16	Il sistema deve mostrare in dettaglio il sistema operativo del dispositivo registrato.	Soddisfatto
RF-Ob17	Il sistema deve mostrare in dettaglio la descrizione del dispositivo registrato.	Soddisfatto
RF-Ob18	Il sistema deve calcolare e mostrare lo stato aggregato di valutazione del dispositivo (non valutato, PASS, FAIL).	Soddisfatto
RF-Ob19	Il sistema deve permettere l'esportazione di tutti i dati di un dispositivo in formato JSON o CSV.	Soddisfatto
RF-Ob20	Il sistema deve consentire l'esportazione dei dati del dispositivo e degli asset _G associati in formato JSON.	Soddisfatto
RF-Ob21	Il sistema deve consentire l'esportazione dei dati del dispositivo e degli asset _G associati in formato CSV.	Soddisfatto
RF-Ob22	Il sistema deve permettere l'eliminazione definitiva di un dispositivo dal sistema.	Soddisfatto
RF-Ob23	Il sistema deve consentire l'eliminazione diretta di un dispositivo senza effettuare il backup dei dati.	Soddisfatto
RF-Ob24	Il sistema deve consentire l'eliminazione del dispositivo previa esportazione automatica di backup dei dati.	Soddisfatto
RF-Ob25	Il sistema deve permettere l'inserimento di un nuovo asset _G all'interno di un dispositivo.	Soddisfatto
RF-Ob26	Il sistema deve richiedere la compilazione dei dati dell'asset _G nel form di creazione.	Soddisfatto
RF-Ob27	Il sistema deve permettere l'inserimento del nome dell'asset _G nel form di creazione.	Soddisfatto
RF-Ob28	Il sistema deve permettere la selezione del tipo di asset _G tra Network, Security, Privacy e Financial.	Soddisfatto
RF-Ob29	Il sistema deve permettere l'inserimento della descrizione dell'asset _G nel form di creazione.	Soddisfatto

Codice	Descrizione	Stato
RF-Ob30	Il sistema deve permettere di impostare la sensibilità dell'asset _G .	Soddisfatto
RF-Ob31	Il sistema deve permettere la visualizzazione della lista degli asset _G associati ad un determinato dispositivo.	Soddisfatto
RF-Ob32	Il sistema deve mostrare le informazioni essenziali del singolo asset _G all'interno della lista.	Soddisfatto
RF-Ob33	Il sistema deve mostrare il nome del singolo asset _G all'interno della lista.	Soddisfatto
RF-Ob34	Il sistema deve mostrare il tipo del singolo asset _G all'interno della lista.	Soddisfatto
RF-Ob35	Il sistema deve mostrare lo stato di valutazione del singolo asset _G all'interno della lista.	Soddisfatto
RF-Ob36	Il sistema deve permettere la visualizzazione in dettaglio di tutte le informazioni di un singolo asset _G selezionato.	Soddisfatto
RF-Ob37	Il sistema deve mostrare nel dettaglio il nome dell'asset _G selezionato.	Soddisfatto
RF-Ob38	Il sistema deve mostrare nel dettaglio il tipo dell'asset _G selezionato.	Soddisfatto
RF-Ob39	Il sistema deve mostrare nel dettaglio la descrizione dell'asset _G selezionato.	Soddisfatto
RF-Ob40	Il sistema deve mostrare nel dettaglio il livello di sensibilità dell'asset _G selezionato.	Soddisfatto
RF-Ob41	Il sistema deve mostrare lo stato complessivo di valutazione dell'asset _G selezionato.	Soddisfatto
RF-Ob42	Il sistema deve mostrare la lista dei requisiti _G da valutare associati all'asset _G .	Soddisfatto
RF-Ob43	Il sistema deve mostrare il codice identificativo e lo stato di valutazione di ogni requisito nella lista.	Soddisfatto
RF-Ob44	Il sistema deve permettere l'eliminazione definitiva di un asset _G da un dispositivo.	Soddisfatto
RF-Ob45	Il sistema deve permettere l'esecuzione di una sessione di	Soddisfatto
RF-Ob46	Il sistema deve mostrare una dashboard _G di valutazione con la lista degli asset _G , il loro stato, e il progresso della sessione in tempo reale.	Soddisfatto
RF-Ob47	Il sistema deve consentire la selezione e l'avvio della valutazione dei requisiti _G di un singolo asset _G .	Soddisfatto
RF-Ob48	Il sistema deve mostrare il nome, il tipo, la descrizione, la sensibilità e lo stato di valutazione dell'asset _G selezionato per la valutazione.	Soddisfatto

Codice	Descrizione	Stato
RF-Ob49	Il sistema deve mostrare la lista dei requisiti _G associati all'asset _G in valutazione.	Soddisfatto
RF-Ob50	Il sistema deve mostrare, per ciascun requisito nella lista, il codice e lo stato di valutazione.	Soddisfatto
RF-Ob51	Il sistema deve mostrare il codice e il nome del requisito selezionato prima dell'avvio dell'esecuzione del decision tree _G .	Soddisfatto
RF-Ob52	Il sistema deve mostrare le dipendenze del requisito selezionato e il loro stato di valutazione prima dell'esecuzione.	Soddisfatto
RF-Ob53	Il sistema deve guidare l'utente eseguendo il decision tree _G associato al requisito selezionato.	Soddisfatto
RF-Ob54	Il sistema deve mostrare il codice univoco e il testo della domanda del nodo _G corrente dell'albero.	Soddisfatto
RF-Ob55	Il sistema deve registrare la risposta dell'utente avanzandone il percorso sul grafo.	Soddisfatto
RF-Ob56	Il sistema deve gestire la risposta affermativa («Yes») spostando il flusso sul relativo ramo.	Soddisfatto
RF-Ob57	Il sistema deve gestire la risposta negativa («No») spostando il flusso sul relativo ramo.	Soddisfatto
RF-Ob58	Il sistema deve visualizzare a schermo il grafo completo del decision tree _G durante l'esecuzione.	Soddisfatto
RF-Ob59	Il sistema deve evidenziare graficamente nel grafo il nodo _G corrente e il percorso già intrapreso.	Soddisfatto
RF-Ob60	Il sistema deve mostrare un nodo _G foglia al termine del percorso con l'esito (PASS, FAIL, NOT APPLICABLE).	Soddisfatto
RF-Ob61	Il sistema deve generare un file JSON contenente lo stato della sessione di valutazione per il download.	Soddisfatto
RF-Ob62	Il sistema deve permettere il caricamento di un file di sessione per riprendere un test interrotto.	Soddisfatto
RF-Ob63	Il sistema deve mostrare una schermata finale con il riepilogo complessivo di tutti gli esiti del test.	Soddisfatto
RF-Ob64	Il sistema deve mostrare per ogni asset _G la lista dei requisiti _G completati e il percorso logico seguito.	Soddisfatto
RF-Ob65	Il sistema deve mostrare la sequenza ordinata di domande e risposte fornite per un requisito completato.	Soddisfatto
RF-Ob66	Il sistema deve mostrare l'elenco dei decision tree _G disponibili.	Soddisfatto
RF-Ob67	Il sistema deve mostrare l'ID e il nome del requisito per ogni decision tree _G in elenco.	Soddisfatto

Codice	Descrizione	Stato
RF-Ob68	Il sistema deve permettere la visualizzazione in dettaglio di un decision tree _G esistente, mostrandone l'identificativo e il nome del requisito associato.	Soddisfatto
RF-Ob69	Il sistema deve permettere all'utente di uscire anticipatamente da una sessione di valutazione in corso.	Soddisfatto
RF-Ob70	Il sistema deve mostrare il riepilogo degli esiti per ogni singolo asset _G al termine del test.	Soddisfatto
RF-Ob71	Il sistema deve permettere il salvataggio della sessione di valutazione in corso, generando un file con lo stato della sessione.	Soddisfatto
RF-Ob72	Il sistema deve visualizzare il grafo del decision tree _G nel dettaglio, mostrando nodi interni, nodi foglia con esito e collegamenti fra nodi.	Soddisfatto
RF-Ob73	Il sistema deve mostrare i nodi interni del grafo del decision tree _G , con il relativo codice univoco e il testo della domanda.	Soddisfatto
RF-Ob74	Il sistema deve mostrare i nodi foglia del grafo del decision tree _G , con l'esito associato (PASS, FAIL, NOT APPLICABLE).	Soddisfatto
RF-Ob75	Il sistema deve mostrare i collegamenti fra i nodi del grafo del decision tree _G , con l'etichetta Yes/No associata a ciascun ramo.	Soddisfatto
RF-Ob76	Il sistema deve mostrare le dipendenze del decision tree _G , elencando i requisiti _G da cui esso dipende con il relativo codice.	Soddisfatto
RF-Ob77	Il sistema deve permettere l'esportazione di un file di un decision tree _G in formato JSON o CSV.	Soddisfatto
RF-Ob78	Il sistema deve consentire l'esportazione di un decision tree _G in formato JSON.	Soddisfatto
RF-Ob79	Il sistema deve consentire l'esportazione di un decision tree _G in formato CSV.	Soddisfatto
RF-Ob80	Il sistema deve generare un report di conformità finale contenente, per ogni coppia asset _G -requisito, l'esito del requisito, l'esito aggregato del decision tree _G e il percorso logico seguito.	Soddisfatto

Copertura: 80 requisiti_G soddisfatti su 80 (100%).

Il requisito non soddisfatto è: RF-Ob80, relativo alla generazione del report di conformità finale. La funzionalità è progettata ma non ancora realizzata: la sua collocazione architetturale e l'interpretazione di «esito aggregato del decision tree_G» sono punti aperti, discussi in Sezione 3.1.1.

6.2. Requisiti_G funzionali desiderabili

Introducono funzionalità a valore aggiunto che migliorano la fluidità del flusso operativo, senza costituire condizione bloccante per l'utilizzo del prodotto.

Codice	Descrizione	Stato
RF-D01	Il sistema deve permettere l'annullamento della procedura di inserimento di un dispositivo, ripristinando lo stato precedente.	Soddisfatto
RF-D02	Il sistema deve permettere l'annullamento della procedura di modifica di un dispositivo, scartando i dati inseriti e mantenendo invariati quelli preesistenti.	Soddisfatto
RF-D03	Il sistema deve permettere l'annullamento della procedura di eliminazione di un dispositivo durante la fase di richiesta di conferma.	Soddisfatto
RF-D04	Il sistema deve permettere l'annullamento della procedura di inserimento di un asset _G , ripristinando lo stato precedente.	Soddisfatto
RF-D05	Il sistema deve permettere l'annullamento della procedura di modifica di un asset _G , scartando le modifiche non salvate.	Soddisfatto
RF-D06	Il sistema deve permettere l'annullamento della procedura di eliminazione di un asset _G durante la fase di richiesta di conferma.	Soddisfatto
RF-D07	Il sistema deve permettere la navigazione al nodo _G precedente del decision tree _G , mostrando la risposta già fornita senza invalidare le risposte successive.	Soddisfatto
RF-D08	Il sistema deve permettere il salvataggio intermedio dello stato della sessione di valutazione.	Soddisfatto
RF-D09	Il sistema deve permettere all'utente di navigare verso il nodo _G successivo precedentemente già risposto durante l'esecuzione del decision tree _G .	Non soddisfatto
RF-D10	Il sistema deve permettere la modifica della risposta a un nodo _G già risposto, invalidando le risposte successive al nodo _G corrente.	Soddisfatto
RF-D11	Il sistema deve permettere la modifica delle informazioni anagrafiche di un dispositivo esistente.	Soddisfatto
RF-D12	Il sistema deve consentire la modifica del nome del dispositivo.	Soddisfatto
RF-D13	Il sistema deve consentire la modifica del sistema operativo del dispositivo.	Soddisfatto
RF-D14	Il sistema deve consentire la modifica della descrizione del dispositivo.	Soddisfatto
RF-D15	Il sistema deve permettere la modifica delle informazioni di un asset _G esistente.	Soddisfatto
RF-D16	Il sistema deve consentire la modifica del nome dell'asset _G .	Soddisfatto
RF-D17	Il sistema deve consentire la modifica del tipo dell'asset _G tramite opzioni predefinite.	Soddisfatto

Codice	Descrizione	Stato
RF-D18	Il sistema deve consentire la modifica della descrizione dell'asset _G .	Soddisfatto
RF-D19	Il sistema deve consentire la modifica della sensibilità dell'asset _G .	Soddisfatto
RF-D20	Il sistema deve permettere l'importazione e la validazione strutturale di un nuovo decision tree _G da file.	Soddisfatto
RF-D21	Il sistema deve permettere la selezione del file sorgente per l'importazione di un decision tree _G .	Soddisfatto
RF-D22	Il sistema deve supportare l'importazione di un decision tree _G da file in formato JSON.	Soddisfatto
RF-D23	Il sistema deve supportare l'importazione di un decision tree _G da file in formato CSV.	Soddisfatto

Copertura: 22 requisiti_G soddisfatti su 23 (95.7%).

Il requisito non soddisfatto è: RF-D09, relativo alla navigazione verso un nodo_G successivo già risposto. La transizione corrispondente è realizzata nel TreeStore, che espone l'operazione goForward e conserva le risposte successive al cursore, ma il comando non è ancora esposto nell'interfaccia: il requisito è pertanto da considerarsi non soddisfatto dal punto di vista dell'utente.

6.3. Requisiti_G funzionali opzionali

Estendono la piattaforma con funzionalità ausiliarie e di raffinamento, gradite ma non essenziali ai fini della validazione dell'applicazione.

Codice	Descrizione	Stato
RF-Op01	Il sistema deve mostrare la notifica dell'avvenuto salvataggio intermedio della sessione di valutazione.	Non soddisfatto
RF-Op02	Il sistema deve permettere l'aggiunta manuale di una dipendenza tra requisiti _G all'interno di un decision tree _G .	Non soddisfatto
RF-Op03	Il sistema deve permettere la rimozione di una dipendenza tra requisiti _G da un decision tree _G .	Non soddisfatto
RF-Op04	Il sistema deve bloccare l'aggiunta e notificare l'utente se il requisito selezionato crea una dipendenza circolare.	Non soddisfatto
RF-Op05	Il sistema deve permettere l'aggiunta di un nuovo nodo _G all'interno di un decision tree _G .	Non soddisfatto
RF-Op06	Il sistema deve permettere l'inserimento di un codice univoco per il nuovo nodo _G .	Non soddisfatto
RF-Op07	Il sistema deve permettere l'inserimento del testo della domanda del nuovo nodo _G .	Non soddisfatto

Codice	Descrizione	Stato
RF-Op08	Il sistema deve permettere l'eliminazione di un nodo _G esistente da un decision tree _G .	Non soddisfatto
RF-Op09	Il sistema deve impedire la creazione di collegamenti duplicati mostrando una notifica di errore.	Non soddisfatto
RF-Op10	Il sistema deve validare la struttura dell'albero modificato secondo i vincoli di consistenza predefiniti.	Non soddisfatto
RF-Op11	Il sistema deve impedire il salvataggio e mostrare un errore se l'albero non è binario o mancano foglie PASS/FAIL.	Non soddisfatto
RF-Op12	Il sistema deve impedire l'eliminazione del nodo _G radice di un decision tree _G mostrando un errore.	Non soddisfatto
RF-Op13	Il sistema deve consentire l'eliminazione definitiva di un decision tree _G .	Non soddisfatto
RF-Op14	Il sistema deve bloccare l'inserimento e mostrare un messaggio di errore se il codice del nodo _G è già presente nel decision tree _G .	Non soddisfatto
RF-Op15	Il sistema deve permettere all'utente di assegnare un esito (PASS, FAIL o NOT APPLICABLE) ai rami non collegati di un nodo _G appena aggiunto o modificato nel decision tree _G , trasformandoli in nodi foglia.	Non soddisfatto
RF-Op16	Il sistema deve assegnare l'esito PASS al ramo non collegato selezionato dall'utente, creando un nodo _G foglia PASS.	Non soddisfatto
RF-Op17	Il sistema deve assegnare l'esito FAIL al ramo non collegato selezionato dall'utente, creando un nodo _G foglia FAIL.	Non soddisfatto
RF-Op18	Il sistema deve assegnare l'esito NOT APPLICABLE al ramo non collegato selezionato dall'utente, creando un nodo _G foglia NOT APPLICABLE.	Non soddisfatto
RF-Op19	Il sistema deve permettere l'annullamento delle modifiche effettuate su un decision tree _G , ripristinando lo stato iniziale del grafo.	Non soddisfatto
RF-Op20	Il sistema deve permettere la modifica della destinazione di un collegamento tra nodi (Yes/No).	Non soddisfatto
RF-Op21	Il sistema deve permettere la modifica strutturale di un decision tree _G esistente.	Non soddisfatto
RF-Op22	Il sistema deve consentire il download del report di conformità in formato PDF.	Soddisfatto
RF-Op23	Il sistema deve consentire il download del report di conformità in formato JSON.	Non soddisfatto
RF-Op24	Il sistema deve consentire il download del report di conformità in formato CSV.	Non soddisfatto

Codice	Descrizione	Stato
RF-Op25	Il sistema deve mostrare la giustificazione testuale del risultato raggiunto al termine dell'esecuzione del decision tree _G .	Non soddisfatto
RF-Op26	Il sistema deve permettere l'inserimento di una giustificazione testuale per l'esito della coppia asset _G -requisito al termine dell'esecuzione del decision tree _G .	Non soddisfatto

Copertura: 1 requisito soddisfatto su 26 (3.8%).

Il solo requisito soddisfatto della categoria è RF-Op22, download del report in formato PDF. Gli altri requisiti_G opzionali riguardano in prevalenza la modifica strutturale dei decision tree_G, l'esportazione del report negli altri formati e funzionalità accessorie di consultazione. La loro realizzazione è subordinata al completamento dei requisiti_G obbligatori e desiderabili e alla disponibilità di tempo residuo, secondo quanto stabilito in sede di pianificazione.

7. Requisiti_G di sistema

Il prodotto è distribuito come applicazione web locale containerizzata. I requisiti_G riportati in questa sezione si riferiscono pertanto alla macchina che ospita i container e al browser impiegato per accedere all'interfaccia; non è previsto alcun server remoto né alcuna infrastruttura condivisa.

Si distinguono due scenari con esigenze differenti: la costruzione delle immagini, che avviene una sola volta e richiede il collegamento a Internet, e l'esecuzione ordinaria, che avviene interamente in locale.

7.1. Requisiti_G hardware

I valori riportati sono stati rilevati su un'installazione funzionante del prodotto.

Risorsa	Rilevato	Consigliato
Memoria occupata dai container	42 MiB	—
Memoria complessiva della macchina	—	4 GB
Spazio per le immagini	288 MB	—
Spazio complessivo su disco	—	2 GB
Catalogo dei decision tree _G	40 KB	—

L'occupazione di memoria dei due container è di 22 MiB per il backend_G e 20 MiB per il frontend_G. Si mantiene stabile durante l'uso. La ragione è che il servizio di frontend_G si limita a servire file statici tramite Nginx, mentre l'applicazione vera e propria è eseguita dal browser dell'utente. Il consumo di risorse significativo è pertanto quello del browser, non quello dei container, ed è il motivo per cui la memoria complessiva consigliata è di 4 GB in modo da essere sufficiente a ospitare il motore di containerizzazione, il browser e il sistema operativo.

Le immagini prodotte occupano complessivamente 288 MB, di cui 212 MB per il backend_G e 76,1 MB per il frontend_G. Lo spazio consigliato di 2 GB tiene conto dei livelli temporanei generati durante la costruzione delle immagini, in particolare le dipendenze di sviluppo del frontend_G, assenti dall'immagine finale grazie alla costruzione in due fasi e del margine necessario alla crescita del catalogo dei decision tree_G, che l'utente può ampliare tramite importazione.

Il catalogo dei decision tree_G occupa allo stato attuale 40 KB, essendo costituito da documenti JSON di dimensioni ridotte. La sua crescita è lineare nel numero di alberi importati e non pone vincoli di rilievo.

Nessun limite di risorse è dichiarato nel file di orchestrazione: i container utilizzano quanto reso disponibile dal motore di containerizzazione.

7.2. Requisiti_G software

Componente	Versione	Note
Sistema operativo	—	Qualsiasi sistema in grado di eseguire Docker: distribuzioni Linux, macOS, Windows.
Docker Engine	20.10 o successiva	Necessario per la costruzione e l'esecuzione dei container. Su macOS e Windows è fornito da Docker Desktop.
Docker Compose	v2	Impiegato per l'orchestrazione dei due servizi tramite il file <code>docker-compose.yml</code> .
Browser web	—	Necessario per accedere all'interfaccia. I browser supportati sono indicati di seguito.

Sulla macchina ospite non è richiesta l'installazione né di Python né di Node.js: entrambi gli ambienti di esecuzione sono contenuti nelle rispettive immagini. Il backend_G è costruito a partire da `python:3.12-slim`, il frontend_G è compilato con `node:22-alpine` e distribuito su `nginx:1.27-alpine`; l'ambiente di compilazione non è presente nell'immagine finale.

Il requisito di vincolo_G RV-Ob02 stabilisce la compatibilità con le ultime due versioni principali di Google Chrome, Mozilla Firefox, Apple Safari e Microsoft Edge. Il prodotto è realizzato con tecnologie web standard e non impiega funzionalità sperimentali o specifiche di un singolo motore di rendering.

7.3. Requisiti_G di rete

L'esecuzione del prodotto non richiede alcun collegamento a Internet. Client e server comunicano sulla rete privata creata da Docker Compose, e l'interfaccia è raggiungibile dall'ospite all'indirizzo `http://localhost:8080`.

Le porte seguenti devono essere disponibili sulla macchina ospite:

Porta	Servizio	Utilizzo
8080	frontend _G	Accesso all'interfaccia utente. Corrisponde alla porta 80 del container, sulla quale è in ascolto Nginx.
5000	backend _G	Accesso diretto all'API REST. Non necessaria all'uso ordinario, poiché le richieste dell'interfaccia sono inoltrate al backend _G da Nginx sulla rete privata; è esposta per finalità di collaudo e diagnosi.

Il collegamento a Internet è necessario unicamente in fase di **costruzione** delle immagini, per il recupero delle immagini base e delle dipendenze applicative dai rispettivi registri. Una volta costruite, le immagini possono essere eseguite su una macchina isolata dalla rete esterna.

L'assenza di dipendenze da servizi remoti è coerente con la natura del prodotto: i dati dell'utente non transitano né permangono su alcun archivio esterno, e lo scambio di dispositivi, sessioni e decision tree_G fra installazioni avviene mediante esportazione e importazione di file.